

Hierarchical Fault Detection and Diagnosis for Transformer Architectures

SIGMA JAHAN, Dalhousie University, Canada

SAURABHSINGH RAJPUT, Dalhousie University, Canada

TUSHAR SHARMA, Dalhousie University, Canada

MOHAMMAD MASUDUR RAHMAN, Dalhousie University, Canada

Transformer models are widely deployed in critical AI applications, yet faults in their attention mechanisms, projections, and other internal components can degrade behavior silently without raising runtime errors. Existing fault diagnosis techniques often target generic deep neural networks and cannot identify which transformer component is responsible for an observed symptom. In this article, we present DEFault++, a hierarchical learning-based diagnostic technique that operates at three levels of abstraction. DEFault++ detects whether a fault is present, classifies it into one of 12 transformer-specific fault categories (covering both attention-internal mechanisms and surrounding architectural components), and identifies the underlying root cause from up to 45 mechanisms. To facilitate both training and evaluation, we construct DEFault-bench, a benchmark of 5,556 labeled instances obtained through systematic mutation testing. We create these instances across seven transformer models and nine downstream tasks using DEForm, a transformer-specific mutation technique we developed for this purpose. DEFault++ measures runtime behavior at the level of individual transformer components. It organizes these measurements through a Fault Propagation Graph (FPG) derived from the transformer architecture. It then produces an interpretable diagnosis using prototype matching combined with supervised contrastive learning. On DEFault-bench, DEFault++ achieves an F1 of 0.826–0.909 for detection, 0.850–0.868 for categorization, and 0.851–0.867 for root-cause diagnosis on encoder and decoder architectures. In a developer study with 21 practitioners, the accuracy of choosing correct repair actions increased from 57.1% without support to 83.3% when using DEFault++.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: attention mechanism, fault diagnosis, fault propagation, mutation testing, transformer

ACM Reference Format:

Sigma Jahan, Saurabhsingh Rajput, Tushar Sharma, and Mohammad Masudur Rahman. 2026. Hierarchical Fault Detection and Diagnosis for Transformer Architectures. 1, 1 (May 2026), 55 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Transformer models are widely used across natural language processing [22, 70], code generation [10], software engineering [74], and medical imaging [19]. Their effectiveness stems largely from the attention mechanism [89], which addresses limitations of sequential models such as Recurrent Neural Networks (RNNs) [89]. Attention is implemented through model code, optimized kernels, mixed-precision arithmetic, and task-specific preprocessing. Small implementation mistakes in attention logic, masking, or kernel integration can silently alter model behavior without raising

Authors' Contact Information: Sigma Jahan, Dalhousie University, Halifax, Canada, sigma.jahan@dal.ca; Saurabhsingh Rajput, Dalhousie University, Halifax, Canada, saurabh@dal.ca; Tushar Sharma, Dalhousie University, Halifax, Canada, tushar@dal.ca; Mohammad Masudur Rahman, Dalhousie University, Halifax, Canada, masud.rahman@dal.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

explicit errors [14, 38, 63, 90, 98, 105]. For example, attention heads can degenerate or become redundant [62, 90]. These faults can reduce representation diversity and affect model performance without producing immediate errors. Even when a symptom becomes visible (e.g., poor output quality), it rarely identifies the responsible component. The same symptom can arise from projection errors, masking faults, positional encoding faults, or residual corruption [37]. Developers can inspect loss curves, gradients, attention maps, and parameter updates, but these measurements alone do not connect a visible symptom to the root cause or the responsible transformer component. Diagnosis therefore requires component-level instrumentation, a representation of how a deviation at one component reaches the rest of the block, and a separation step that can discriminate root causes that disturb the same component.

Two senses of the word “causal” appear in this paper. The first is the architectural masking constraint in autoregressive decoders, where attention is blocked from future positions. The second is controlled fault attribution under matched-seed mutation, where each injected fault is paired with a clean baseline. We do not claim formal causal inference in the sense of Pearl-style identifiability or causal-graph discovery.

Recent work has proposed automated debugging techniques for DNN programs. Rule-based techniques, such as AutoTrainer [107], UMLAUT [83], and DeepDiagnosis [94], check training runs against fixed symptoms such as exploding gradients, vanishing gradients, or oscillating loss. Learned classifiers, such as DeepFD [7] and DEFault [39], train on runtime features to predict fault categories such as loss function, learning rate, or activation. Both lines of work are architecture-agnostic and remain useful for general training and model faults in transformer programs. However, they do not address the class of faults that arise from transformer-specific mechanisms [37, 38]. These techniques observe runtime behavior at the model level rather than inside individual transformer components. As a result, they cannot reliably distinguish transformer-specific faults from generic training problems, nor can they identify which component is responsible. Transformer-specific debuggers such as ATTNChecker [49] and AtPatch [99] target narrower failure modes, such as NaN or Inf values during attention or attention-map anomalies at the patch level. They do not cover the full taxonomy of transformer faults (Figure 3). Consequently, existing techniques report broad categories such as *incorrect activation* or *incorrect loss*, without identifying which transformer component caused the fault or which mechanism inside that component explains it.

To address these gaps, we propose DEFault++, a learning-based diagnostic technique for transformer models. DEFault++ produces three levels of diagnosis. Level 1 detects whether a fault is present. Level 2 classifies the fault into its category, such as QKV or masking. Level 3 identifies the root cause within that category. Detection at Level 1 is itself non-trivial for transformer faults, since these faults can present as silent failures that leave loss curves and finite values unchanged while internal attention behavior shifts. Our key insight is that each transformer fault can leave a distinctive pattern inside the affected component, even when the overall training metrics appear normal. DEFault++ measures how each transformer component behaves during training and connects these measurements through a Fault Propagation Graph (FPG) built from the internal structure of the transformer. The FPG is a structural prior derived from the transformer’s forward and backward equations. It organizes the runtime measurements so the diagnostic model can associate observed deviations with a fault category and report the runtime patterns that support each diagnosis. On DEFault-bench, which contains 5,556 labeled instances from encoder and decoder architectures, DEFault++ outperforms four existing DNN debugging techniques on Level 1 fault detection F1. In our developer study (N = 21), repair-action accuracy improved from 57.1% without assistance to 83.3% with DEFault++ assistance. In summary, we make the following contributions:

- (a) DEfault++, a hierarchical technique that detects faults in transformer programs, classifies them into fault categories, and identifies the root cause within that category.
- (b) DEForm, a mutation technique for transformer models based on a taxonomy of real-world transformer faults.
- (c) DEfault-bench, a transformer fault detection benchmark of 5,556 labeled instances produced by DEForm.
- (d) A Fault Propagation Graph (FPG), a structural prior derived from the transformer's forward and backward equations that encodes the dependency paths between transformer components and provides the diagnostic model with the transformer's architectural structure.
- (e) A training objective for root-cause diagnosis that combines supervised contrastive learning with prototype matching to separate root causes within the same fault category, and an FPG-based explanation that reports which feature groups support each diagnosis.
- (f) An evaluation of DEfault++ on real-world transformer faults and through a developer study with 21 participants.

2 Motivating Example

```

1  class Attention(nn.Module):
2      def __init__(self, dim, heads=8):
3          super().__init__()
4          self.dim = dim
5          self.q_proj = nn.Linear(dim, dim, bias=False)
6          self.k_proj = nn.Linear(dim, dim, bias=False)
7          self.v_proj = nn.Linear(dim, dim, bias=False)
8          self.out_proj = nn.Linear(dim, dim)
9          self.to_qkv = None # populated by fuse_qkv_projections
10
11     def fuse_qkv_projections(self):
12         """Fuse Q/K/V into a single linear layer for efficiency."""
13         w = torch.cat(
14             [self.q_proj.weight, self.k_proj.weight, self.v_proj.weight],
15             dim=0,
16         )
17         self.to_qkv = nn.Linear(self.dim, 3 * self.dim, bias=False)
18         with torch.no_grad():
19             self.to_qkv.weight.copy_(w)
20         # fault: q_proj, k_proj, v_proj NOT deleted; LoRA still attaches to them
21
22     def forward(self, x):
23         # fault: q_proj/k_proj/v_proj are bypassed; forward uses fused layer
24         q, k, v = self.to_qkv(x).chunk(3, dim=-1)
25         # ... attention computation with q, k, v ...
26         return self.out_proj(attn_output)
27
28     # Downstream: LoRA targets the stale projections
29     lora_config = LoraConfig(target_modules=["q_proj", "v_proj"]) # inactive in forward
30     model = get_peft_model(pipeline.unet, lora_config)
31     model.train() # LoRA adapts stale layers; no effect on forward pass

```

Listing 2.1. Example of real world transformer fault (HuggingFace Diffusers Issue #11903)

To show the class of faults DEfault++ targets, consider the real-world fault in Listing 2.1, taken from the HuggingFace Diffusers library (Issue #11903¹). The method `fuse_qkv_projections` merges the query, key, and value projections (`q_proj`, `k_proj`, `v_proj`) into one fused layer (`to_qkv`) and routes the forward pass through that fused layer. The original projection modules are kept so that LoRA [27] can still attach to its declared adapter targets, but the forward

¹HuggingFace Diffusers Issue #11903

pass no longer uses them. LoRA therefore updates parameters that no longer affect the model output. The model runs without exception, the loss decreases, and no invalid values (e.g., NaN or Inf) appear. The only observable sign of the fault is that fine-tuning does not change model behavior.

Static analysis misses this fault because the original layers still exist, are correctly typed, and remain reachable in the parameter list. Dynamic techniques miss it for one of two reasons. Rule-based tools (e.g., AutoTrainer, UMLAUT, DeepLocalize) [83, 96, 107] look for symptoms such as oscillating loss, heuristic validation-trend violations, or NaN/Inf, none of which occurs in this case. Learned classifiers (e.g., DeepFD, DEFault) [7, 39] use DNN-level fault categories that do not represent transformer components. ATTNChecker [49] targets temporary NaN/Inf in attention, which this fault does not produce. In contrast, DEFault++ measures runtime traces such as QKV alignment, attention entropy, and projection update activity during training. These features help characterize the faulty component (i.e., the QKV projection path). DEFault++ therefore detects the fault at Level 1, assigns it to the correct category (i.e., QKV) at Level 2, and identifies the root cause (i.e., a stale projection update path) at Level 3.

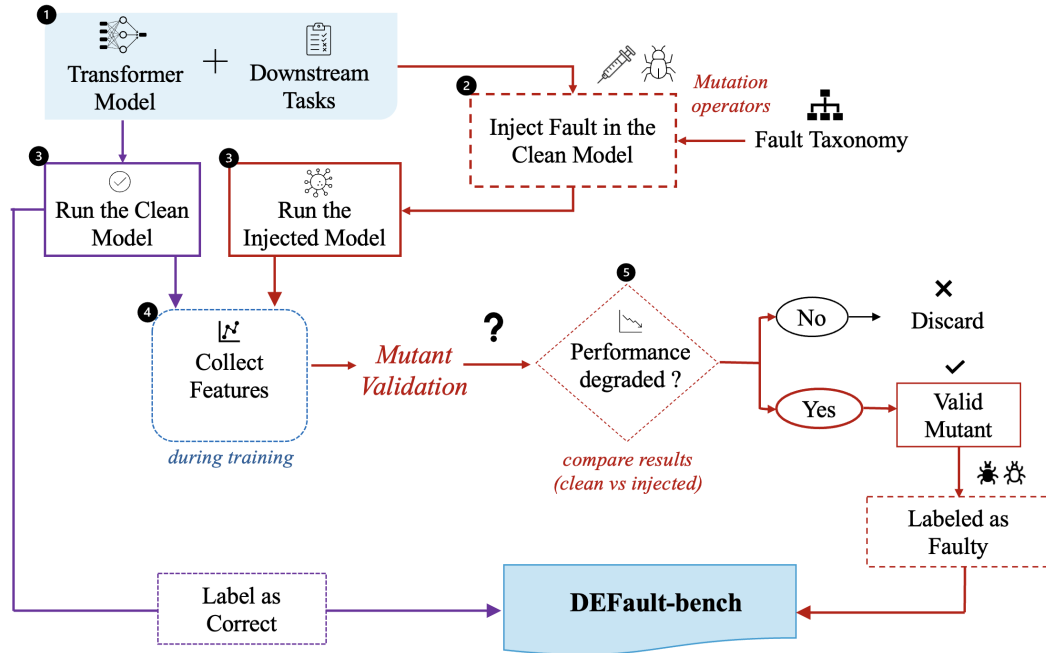


Fig. 1. Workflow for constructing DEFault-bench

3 Benchmark Construction

DEFault-bench is built from clean and faulty fine-tuning runs of seven transformer models across nine downstream tasks. We construct DEFault-bench using *DEForm* (Section 3.2), our mutation technique for transformers. DEForm builds on DeepCrime’s idea of deciding whether a mutation has an effect by repeating the training run several times and applying a statistical test [31]. DeepCrime was designed for generic DNNs and does not cover transformer-specific components. DEForm adds the parts that are missing for transformers. It adds a 12-category fault taxonomy that includes attention-internal components and the decoder KV cache, 52 mutation operators taken from documented fault

root causes, two injection mechanisms (one for stored parameters, one for forward computations), and a structural check that runs before the statistical test. We also replace DeepCrime’s twenty-seed generalized linear model with a one-sided sign-flip permutation test at five matched seeds, which keeps the test exact at $\alpha = 0.05$ and reduces the compute cost per mutant. Each fault is specified by a configuration $C = (m, t, u, f, v, \ell, \sigma)$ that fixes the model m , the fine-tuning task t , the target component u , the fault category f , the fault variant v , the target layer ℓ , and the severity σ . We then run paired clean and faulty fine-tuning under matched random seeds, validate each mutant statistically against its clean counterpart on the task metric, and retain killed mutants together with their training-time measurements as the *faulty* class. We pair these with an equal number of label-preserving clean variants drawn from the same base models to form the *correct* class (see Figure 1 for the complete workflow).

3.1 Fault Taxonomy and Mutation Operators

Each mutation operator matches a transformer fault root cause that has been reported in real-world projects. The attention-internal categories and root causes come from the attention fault taxonomy of Jahan et al. [37], which classifies 555 attention faults collected from open-source projects. The non-attention categories (Embedding, FFN, LayerNorm, Residual, Output) come from prior DNN fault taxonomies that catalog real DL faults: Humatova et al. [30] and Islam et al. [33], which report 970 faults across five deep learning libraries. Following DeepCrime [31], we treat this real-fault link as the validation for each operator. DeepCrime extracted 35 operators from prior real-fault studies and argued that operators derived from real faults do not require a separate realism check. We take the same view. The statistical mutation-killing procedure in Section 3.4 then checks whether each injected fault changes model behavior against a clean baseline, which is separate from whether the fault is realistic. We focus on implementation errors that do not raise runtime errors, such as configuration mistakes and silent numerical issues. These faults require statistical observation against a clean baseline for detection. We exclude explicit faults because conventional tests or runtime checks often detect them. We decompose each transformer block into six *units* that match common implementation boundaries. For a standard transformer block [89]:

$$\mathbf{h} = \text{LayerNorm}(\mathbf{x} + \text{MHA}(\mathbf{x})) \quad (1)$$

$$\mathbf{y} = \text{LayerNorm}(\mathbf{h} + \text{FFN}(\mathbf{h})) \quad (2)$$

where $\mathbf{x} \in \mathbb{R}^{n \times d}$ is an input sequence, MHA is multi-head self-attention, and the position-wise feed-forward network (FFN) follows. From this structure, we define six units: embeddings (input representations), multi-head self-attention, FFN, layer normalization, residual connections, and the output projection head. The same unit boundaries apply under pre-norm variants. Only the ordering of LayerNorm changes. For each unit, we define one or more *fault categories*, and each category contains multiple *variants* that represent specific bug patterns (see Figure 2 for the categories grouped by transformer component). Multi-head self-attention is the most complex unit and a frequent source of faults [38, 62, 90].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \quad (3)$$

where Q, K, V are linear projections of the input, d_k is the key dimension, and M is an additive mask. We treat each component of Equation (3) as a potential fault surface and group attention faults into seven categories. Six apply to both encoders and decoders. The seventh, KV Cache, is decoder-only.

Attention masking faults corrupt the mask M through missing padding masks, inverted mask logic, or incorrect reshaping across batch and head dimensions. For decoder-only self-attention, this category also covers *causal-mask*

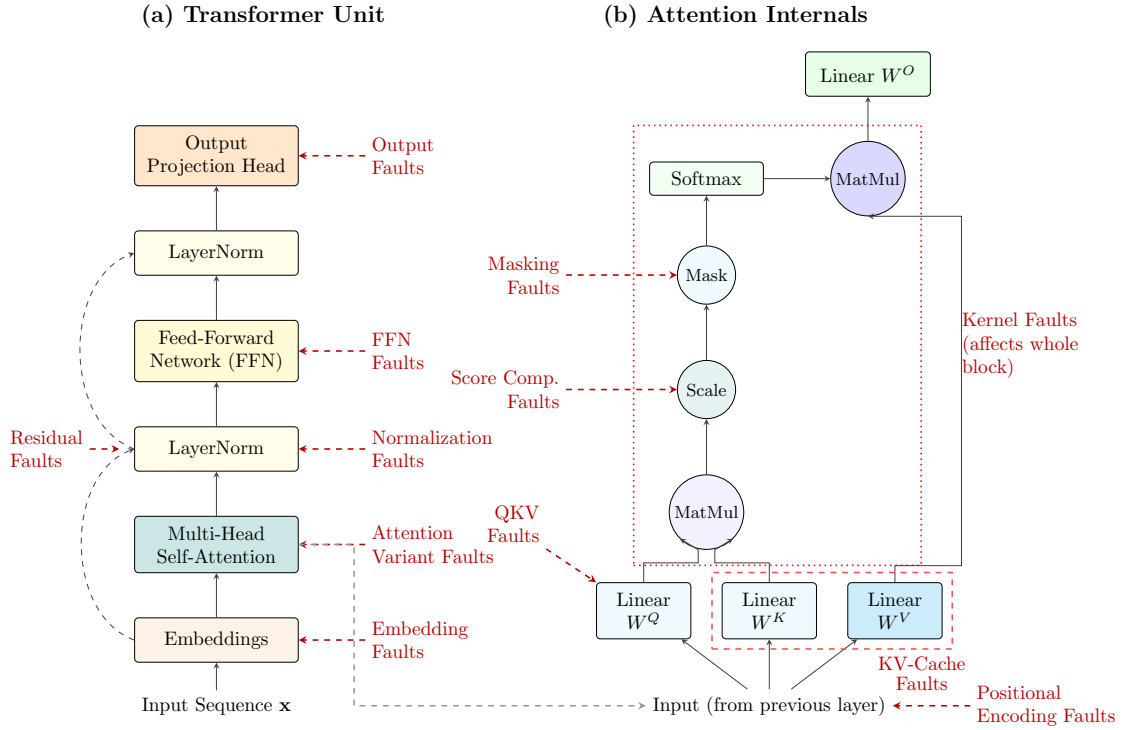


Fig. 2. Fault categories organized by transformer components

violations that allow attention to flow from position i to future positions $j > i$. *QKV projection faults* corrupt the projection matrices W^Q , W^K , W^V through shared or swapped projections, zero-initialized projections, frozen parameters, or misconfigured head-dimension splits. *Score computation faults* corrupt the scaling and dropout around attention scores through missing $1/\sqrt{d_k}$ scaling, misplaced dropout, or unsafe mixed-precision conversions. *Positional encoding faults* corrupt positional information through off-by-one errors, truncation beyond the supported sequence length, or omission of positional embeddings. *Kernel faults* stem from misconfiguration of attention kernels, including mismatched dropout probabilities, silent backend fallbacks, reduced-precision math paths, and memory-constrained execution paths. *Attention variant faults* represent the selection of a wrong attention variant, such as single-head instead of multi-head attention, or applying causal masking in a bidirectional encoder. *KV Cache faults* target key-value cache management in autoregressive decoders through stale caches, desynchronized cache updates, off-by-one indexing, truncated caches, or cross-request leaks [1]. The remaining five units have a single fault category each. *Input embedding faults* corrupt token and segment embeddings by zeroing subsets, swapping pairs, scaling segment embeddings, or zeroing a block of embedding feature dimensions. *FFN faults* alter hidden-layer characteristics by scaling weights, permanently dropping neurons, or replacing the nonlinearity. *Layer normalization faults* modify LayerNorm parameters by scaling or zeroing the learned scale parameter γ , shifting the bias parameter β , misconfiguring the numerical stability term ϵ , or applying weight decay to the LayerNorm parameters [65, 103]. *Residual connection faults* break skip connections by removing them, scaling the residual path, injecting noise, or misconfiguring the residual dropout rate. *Output projection faults* corrupt prediction heads by scaling logits, zeroing rows for selected output classes, or reinitializing the projection.

The complete fault taxonomy defines the Level 2 and Level 3 label spaces (see Figure 3). The seven attention-specific categories and their 25 root causes are derived from the attention fault taxonomy [37]. The five non-attention categories (Input/Embedding, Layer Normalization, FFN, Residual, and Output) and their 20 root causes are derived from prior DNN fault taxonomies [30, 33] and the DeepCrime mutation operators [31]. Each root cause name identifies the fundamental mechanism or parameter that is faulty (e.g., activation function, dropout rate, weight scaling) rather than a specific corrective action. The taxonomy contains 12 fault categories. Eleven apply to encoders and all 12 apply to decoders, since KV Cache is decoder-only. Together, the 12 categories define 40 root causes for encoders and 45 for decoders.

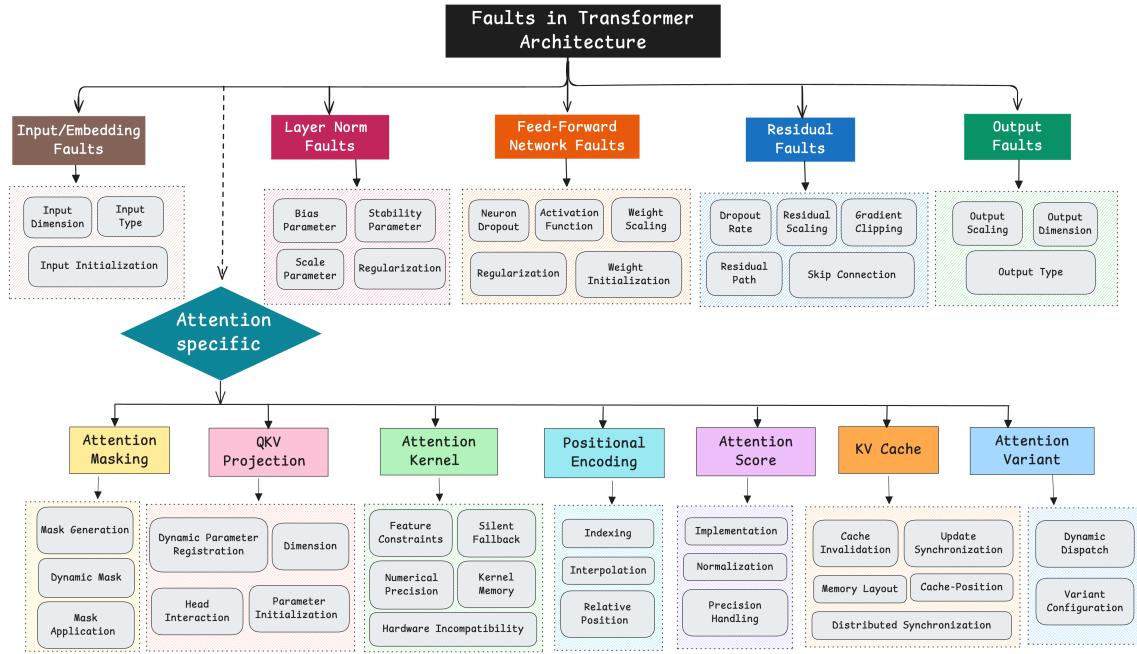


Fig. 3. Taxonomy of transformer fault categories and root-cause labels used by DEFault++

DEForm (Section 3.2) implements 52 mutation operators over this taxonomy, separated into attention-specific operators and architecture-level operators (see Tables 18 and 19 in Section Appendix A). A root cause names a fault mechanism, and an operator is one specific way to inject it. The operator count exceeds the 45 root-cause labels because six root causes are each covered by two or three operators. For example, the QKV parameter-initialization root cause is covered by three operators that zero the query, key, and value projections separately (QZQ, QZK, QZV). Every root cause has at least one operator, and every operator maps to exactly one root-cause label. The Level 3 label space therefore remains 40 root causes for encoders and 45 for decoders. Each operator is based on a transformer fault root cause associated with the units in Figure 2, following prior fault taxonomies [30, 31, 33, 38]. The operator name preserves the corresponding root-cause mechanism, following [31]. However, the final benchmark includes only the configuration that can run successfully during training and evaluation.

Some root causes can lead to either explicit failures or silent behavioral faults, depending on how they are parameterized. For example, changing an output dimension may cause an immediate shape error if the output head no longer matches the task loss. The same root cause may also keep the model runnable while corrupting the output mapping. We discard configurations that crash, produce invalid tensor shapes, call unsupported backend functions, or create

out-of-range indices. The remaining configurations must pass structural verification, complete feature extraction, and produce the required training traces. Each operator has a three-letter ID. The first letter identifies the transformer component (E = Embedding, M = Masking, Q = QKV, S = Score, P = Positional, K = Kernel, V = Variant, C = KV Cache, F = FFN, N = LayerNorm, R = Residual, O = Output). The remaining two letters are a short mnemonic for the operator. For most operators they denote the action and its target, such as QZQ for “in the QKV component, **Z**ero the **Q** projection” and MZM for “**Z**ero the attention **M**ask in the **M**asking component.” For operators whose mechanism is more naturally described by a modifier and an action, the two letters denote the modifier and the action, such as VSH for a Variant operator that leaves a **S**ingle effective **H**ead, VEC for a Variant operator that applies a causal mask in the **E**n**C**oder, and SPD for a Score operator that applies **P**re-softmax **D**ropout (see the Operator and Executable Mutation Parameters columns of Tables 18 and 19 for the full description of each operator).

The Search Type (ST) column says how DEForm (Section 3.2) chooses values for an operator’s parameter. *B* means the operator has no parameter and is either applied or not (e.g., MZM zeroes the attention mask). *EU* means the operator takes a numeric parameter and DEForm runs it once at each value in a list we provide (e.g., ETZ zeroes a configurable percentage of token embeddings). *EL* means the operator takes a choice from a fixed categorical set and DEForm runs it once at each item (e.g., FCA replaces the activation function with each of {ReLU, GELU, Tanh, Sigmoid}).

3.2 DEForm: Fault Injection Mechanism

We denote the clean model parameters by θ and the injected mutant by θ' . We evaluate θ and θ' under matched random seeds and otherwise identical fine-tuning conditions to isolate the causal effect of C . The components of the configuration $C = (m, t, u, f, v, \ell, \sigma)$ are the model architecture, the downstream task, the target unit, the fault category, the fault variant, the affected layer indices $\ell \subseteq \{1, \dots, L\}$ where L is the number of transformer blocks, and the severity level $\sigma \in \{\text{low}, \text{medium}, \text{high}\}$. The severity controls the magnitude of the injected change for numeric variants (e.g., scaling factors, dropout rates) and maps to a discrete intensity for non-numeric variants (e.g., visibility ratio in causal-mask breaking). We limit to single-fault configurations only (i.e., one target unit, one category, and one variant per execution) in order to preserve clarity of isolation and avoid confounding interactions among multiple simultaneous faults. Transformer faults affect either stored parameters or runtime computations, which requires two distinct injection mechanisms [31, 56]. *Static faults* modify parameter tensors at rest, before forward execution. For example, an FFN weight-scaling fault multiplies the targeted weight matrices by a scalar:

$$\theta'_i = \begin{cases} \alpha \cdot \theta_i & \text{if } i \in \text{params}(u, \ell), \\ \theta_i & \text{otherwise} \end{cases} \quad (4)$$

where $\text{params}(u, \ell)$ selects parameters of unit u in layers ℓ . Static faults include embedding corruption, parameter scaling, permanent neuron dropout, and LayerNorm parameter changes. *Dynamic faults* modify the forward computation path through wrappers or hooks at selected module call sites. For example, a mask-zeroing fault intercepts the attention call and replaces the mask M by zeros:

$$\text{forward}'(x, M) = \text{forward}(x, \mathbf{0}_{|M|}) \quad (5)$$

where $\mathbf{0}_{|M|}$ denotes a zero tensor of the same shape as the original mask M . Dynamic faults include mask manipulation, Q/K swapping, removal of score scaling, and forced kernel selection. In both cases, the transformer architecture is

unchanged. Only the targeted computation or parameter values are modified. Figure 4 contrasts the clean and injected execution paths for one representative example of each mechanism.



Fig. 4. Clean and injected version of the same model. Panels (b,d) show static parameter mutation & dynamic forward wrapping

Decoder-only self-attention is causal by construction. The clean model enforces that token position i cannot attend to future positions $j > i$. To inject a causal violation, we weaken or remove the existing constraint to let queries attend to future positions. We do not add an additional causal mask. We replace a configurable fraction of the $-\infty$ entries in the upper-triangular region of the additive mask M with 0, controlled by the severity parameter σ . Each query can then attend to a proportion of keys beyond its own position. We treat causal violations as decoder instances of attention masking faults. To isolate the effect of an injected fault, we keep the main experimental settings identical between θ and θ' runs (e.g., dataset and split assignment, optimizer and schedule, epoch budget, evaluation setup, logging instrumentation, random seed). Each faulty training run is paired with a clean training run under the same seed and otherwise identical training conditions. We implement fault injection with a context manager. The manager stores the original parameter tensors and forward methods before mutation, applies the fault specified by C , and restores the original state after evaluation. The context-manager range prevents carryover effects across configurations.

Before any statistical comparison, we apply a structural verification check. For static faults, we verify that the parameter difference is restricted to $\text{params}(u, \ell)$ and that its magnitude matches σ within a relative tolerance of 10^{-6} to account for floating-point rounding. For dynamic faults, we verify that hooks or wrappers attach only at the intended module call sites and that the original forward function is restored after the context manager exits. We also verify execution completeness by confirming that each configuration produces the required instrumentation logs. Configurations that fail any check are excluded as structurally invalid.

3.3 Subject Models and Tasks

We use four encoder-only and three decoder-only transformer models spanning 66M–125M parameters, 6–12 layers, and both masked and causal language modeling objectives [5, 15, 51, 75, 82]. We limit models to 125M parameters to stay within our compute budget. Encoders are fine-tuned on five GLUE classification tasks [91] (accuracy as task metric), and decoders are fine-tuned on four language-modeling corpora [23, 58, 61, 71] (log-perplexity as task metric). All models use the standard Hugging Face Transformers [29] implementations and default tokenizers, and all seven share the standard transformer-block structure (Equation (3), Figure 2) that the DEForm operators target (see Table 1 for the full model and task list).

Table 1. Models and tasks used to construct DEForm-bench

(a) Subject models

Encoder			Decoder		
Model	Parameter	Layers	Model	Parameter	Layers
BERT-base [15]	110M	12	GPT-2 [75]	124M	12
DistilBERT [82]	66M	6	DistilGPT-2 [82]	82M	6
RoBERTa-base [51]	125M	12	GPT-Neo-125M [5]	125M	12
DistilRoBERTa [82]	82M	6	—	—	—

(b) Fine-tuning tasks

Setting	Tasks	Metric
Encoder (GLUE classification) [91]	SST-2, QNLI, RTE, MRPC, QQP	Accuracy
Decoder (language modeling)	LAMBADA [71], PTB [58], WikiText-2 [61], OpenWebText [23]	Log-perplexity

GLUE tasks: SST-2 (sentiment), QNLI (inference), RTE (entailment), MRPC (paraphrase), QQP (duplicate question).

We fine-tune each model with a single shared setup across tasks (AdamW [52] with linear warmup, mixed-precision) [15, 65] so that observed differences can be attributed to the injected fault rather than to hyperparameter variation. We run each configuration under five fixed seeds $\mathcal{S} = \{42, 123, 456, 789, 101112\}$ [65], the smallest design that supports an exact one-sided sign-flip permutation test at $\alpha = 0.05$ (smallest attainable p -value $1/2^5 \approx 0.031$). We sample layer indices and severity levels low, medium, high uniformly to avoid concentration on any single layer or magnitude. After structural verification, we obtain 4,096 single-fault configurations: 1,955 for encoders and 2,141 for decoders, with all categories represented. Mutant validation required 40,960 paired clean and faulty fine-tuning runs, and the label-preserving clean variants added 13,890 further runs, for a total of 54,850 fine-tuning runs ($\sim 27,000$ GPU-hours on NVIDIA A100 and H100 GPUs). The replication package contains the full hyperparameter list [36] (see ?? for the shared fine-tuning configuration).

3.4 Mutant Validation

We adopt the statistical mutation-killing framework of Humpatova et al. [31]. The original network N and a mutant M are each retrained n times under matched random seeds, and the resulting performance distributions $A_N(\text{TestS}) = \langle A_{N_1}, \dots, A_{N_n} \rangle$ and $A_M(\text{TestS}) = \langle A_{M_1}, \dots, A_{M_n} \rangle$ are compared on a held-out test set TestS . The predicate *isKilled* decides whether the mutant is killed:

$$\text{isKilled}(N, M, \text{TestS}) = \begin{cases} \text{true} & \text{if } p\text{-value}(A_N(\text{TestS}), A_M(\text{TestS})) < \alpha, \\ \text{false} & \text{otherwise.} \end{cases} \quad (6)$$

We use accuracy as the test-set performance metric for encoder classification tasks and log-perplexity for decoder language modeling tasks [38]. We set $n = 5$ matched seeds and $\alpha = 0.05$. Humberova et al. [31] use a generalized linear model with $n = 20$ retrainings paired with Cohen’s d effect size. We instead use a one-sided paired sign-flip permutation test [24] on the per-seed deltas, since seed-paired deltas are exchangeable under the null and five matched seeds are the smallest design that supports an exact one-sided test at $\alpha = 0.05$. The smallest attainable p -value with five paired comparisons is $1/2^5 \approx 0.031$. We estimate false positives by applying *isKilled* to pairs of clean-vs-clean runs. Following Humberova et al. [31], we summarize each mutation operator’s effectiveness using the mutation score:

$$MS(MO) = \frac{|\{c \in MO : isKilled(N, M_c, TestS)\}|}{|MO|} \quad (7)$$

where MO is the set of injected configurations of a mutation operator. The overall mutation score is the average of $MS(MO)$ across all operators. For each killed mutant we attach a label $y = (u, f, v, \ell, \sigma)$ that records the target unit, fault category, fault variant, affected layer set, and severity. We omit m and t because each instance is computed within one model–task context. A surviving mutant, for which *isKilled* returns false, is an injected fault whose measured effect does not pass the killing test ($p > \alpha$). Its label therefore cannot be inferred reliably. We therefore discard surviving mutants and draw the *correct* class from clean (unmutated) base models with label-preserving perturbations, as described in Section 3.5 and summarized per configuration in Algorithm 2. We sample severity uniformly across low, medium, high to avoid centering the benchmark on overly aggressive faults. Severe mutants are often easy to detect due to obvious failures, but they provide limited evidence for distinguishing fault categories [31]. In mutation testing, such mutants are considered *trivial* because their effects are so large that almost any training run reveals them [31].

3.5 Benchmark Statistics

We applied *isKilled* (Equation (6)) to all 4,096 structurally valid configurations and computed the mutation score (Equation (7)) per category and per architecture, following Humberova et al. [31]. Across individual mutants, *isKilled* is true for 1,598 of 1,955 encoder configurations (81.7%) and 1,180 of 2,141 decoder configurations (55.1%), with the per-category killed and surviving counts in Table 2 and the per-category mutation scores in Figure 5. Encoder mutation scores are consistently high, with Variant at 100% and Score, QKV, Positional, FFN, Residual, LayerNorm, and Embedding all above 80%. Kernel, Output, and Masking show the lowest encoder mutation scores (51.5%, 56.8%, and 56.9%). Decoder mutation scores are more variable, both across categories and across model–task pairs (see Figure 15 in Section Appendix C for the per-(model, task) breakdown). Masking and Kernel reach the highest values (77.4% and 77.3%), while Score shows the lowest (25.3%). The low Score rate is consistent with the softmax limiter on the Score-to-Attention edge. Softmax maps a moderate pre-softmax logit shift into a sum-to-one distribution, which can reduce the change before it reaches attention weights and keep the downstream perplexity shift below the detection floor. The faulty and correct classes come from separate, well-defined processes, which removes the equivalent-mutant confound, although this choice excludes some potentially hard negative cases.

The unmutated base model is the only instance whose label is directly known rather than inferred. We therefore draw the *correct* class from these base models. Using a single instance per base model would let a classifier learn model-specific artifacts rather than fault-related patterns. To avoid this risk, we generate label-preserving variants of each base model. We hold the base model fixed and vary only factors that should not change task behavior. These factors include re-running fine-tuning under different random seeds, re-shuffling the training data order where applicable, and varying hyperparameters within ranges that do not shift behavior beyond the significance threshold defined in

Equation (6). The clean variants are behaviorally distinct but semantically equivalent. This variation represents noise that a fault detector should ignore.

For each base model B that produces k killed mutants, we generate k clean variants $\{B_i\}$ from the same base model using these label-preserving perturbations. We evaluate every clean variant against B using the same significance test we use to identify killed mutants. We retain clean variants that remain statistically indistinguishable from B as *correct* samples, and we discard any clean variant that satisfies the killed criterion. This rule makes construction symmetric across the two classes. We also stratify by base model so that the faulty-to-correct ratio stays balanced within each stratum. Our benchmark includes 5,556 instances (Table 2), with 3,196 encoder and 2,360 decoder instances.

Table 2. Composition of DEFault-bench

Category	Variant	Score	QKV	Pos.	FFN	Residual	LayerNorm.	Emb.	Out.	Mask	Kernel	KV
Encoder	25	269	237	166	254	114	181	147	113	41	51	—
Decoder	23	40	78	137	190	112	159	144	74	137	68	18

Note: Encoder contains 1,598 faulty and 1,598 correct samples (total = 3,196). Decoder contains 1,180 faulty and 1,180 correct samples (total = 2,360).

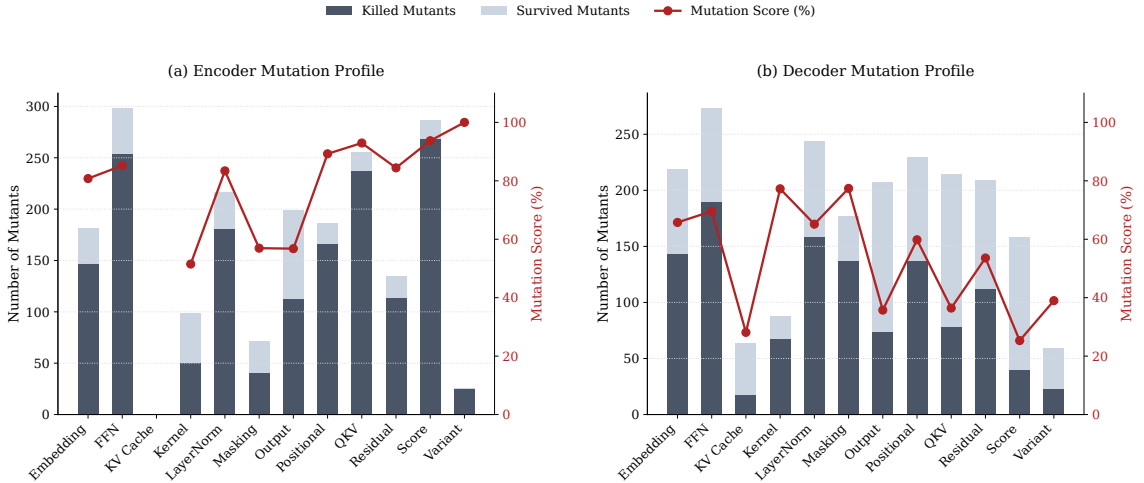


Fig. 5. Per-category mutation scores for encoder and decoder mutants in DEFault-bench

Each instance carries labels at three levels of granularity. At Level 1 (*fault detection*), the label is binary, *faulty* or *correct*. At Level 2 (*fault categorization*), the label is the injected fault category. Encoders use 11 categories: Score, Positional, FFN, QKV, LayerNorm, Residual, Masking, Embedding, Kernel, Output, and Variant. Decoders use the same 11 plus KV Cache. At Level 3 (*root-cause diagnosis*), the label is the injected root cause within the fault category. Each category contains 2–5 root causes, and each root cause belongs to exactly one fault category.

4 DEFault++ Diagnostic Technique

DEFault++ takes as input the labeled set of 5,556 averaged instances from the benchmark constructed in Section 3, where each instance records the runtime measurements collected for one configuration. The three-level hierarchy from Section 1 is realized as follows (see Figure 9). Level 1 uses the full feature representation for binary fault detection. Only instances predicted as faulty proceed to Level 2, which assigns one of the architecture-specific fault categories (11 for

encoders, 12 for decoders). Level 3 predicts the root cause within the Level 2 predicted category through prototypical classification [85] in an embedding space aligned with the transformer’s Fault Propagation Graph. The same embedding space also produces feature-group importance scores for each diagnosis.

4.1 Fault Propagation Graph

The fault categories and root causes defined in Section 3 identify *where* a fault originates, but they do not describe how a fault in one component affects other components during forward computation or backward gradient flow. A stale QKV projection, for example, not only corrupts the projection output but also shifts downstream attention scores, attention weights, and the residual stream. To capture these inter-component dependencies, we define the Fault Propagation Graph (FPG), a directed graph whose nodes are transformer components and whose edges are the forward data-flow dependencies between them. The FPG serves two purposes in DEFault++: it structures the feature representation by mapping feature groups to transformer components (Section 4.2), and it provides the adjacency matrix for message passing in the diagnostic model (Section 4.3.2). Prior DL fault-diagnosis techniques use graph representations for different purposes. NeuralInt [68] encodes a typed meta-model graph for pre-training structural checks, while FL4Deep [64] ranks root causes using a static and dynamic knowledge graph. In contrast, the FPG is a structural prior that encodes the inter-component dependency paths defined by the transformer’s computation graph (see Section 10 for a fuller contrast with prior graph-based representations). We test whether the FPG structure matters in RQ₃ (Section 5.6) by comparing it against rewired and random graphs of the same density.

The FPG represents the forward dependency mechanisms as unlabeled directed edges. The graph is deterministic since it mirrors the fixed computational graph of the transformer. Each edge marks a structural dependency along which a perturbation can travel. The edge does not assert that a specific fault at the source produces an observable change at the target. Whether a perturbation produces an observable downstream effect depends on its magnitude, on intervening nonlinear operations, and on the parameter values learned during training. The FPG is therefore a structural prior. Its edges, derived from the forward pass, group the feature measurements by transformer component and supply the message-passing structure for the diagnostic model.

We define the FPG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each node $v \in \mathcal{V}$ represents a transformer component. The component types are embedding, positional encoding, QKV projection, score computation, attention masking, attention weights, attention output, residual connections, LayerNorm, FFN, and output head. Residual connections and LayerNorm appear independently for each sublayer (attention and FFN). Decoders additionally include a KV cache node. Each directed edge $(v_i, v_j) \in \mathcal{E}$ marks a forward data-flow dependency derived from the transformer’s forward equations, residual structure, and autoregressive state reuse, so an edge means that a perturbation at v_i has a structural path to v_j . The FPG models only transformer components and their dependencies. Not all diagnostic feature groups are FPG nodes. Model-wide context groups and the cross-layer observation group participate in the group-level graph through self-loops only (Table 3). By analyzing the transformer’s forward and backward equations, we identify seven dependency mechanisms. Let δx denote a perturbation to variable x .

- (1) **M1: Forward sequential propagation.** If $B = f(A)$, then $\delta B = (\partial f / \partial A) \delta A$. Perturbation propagates along the data flow path: embedding $\rightarrow$ projections $\rightarrow$ scores $\rightarrow$ masking $\rightarrow$ weights $\rightarrow$ output $\rightarrow$ residual $\rightarrow$ LayerNorm $\rightarrow$ FFN $\rightarrow$ residual $\rightarrow$ output head.
- (2) **M2: Simultaneous propagation.** QKV projections feed both score computation (via Q, K) and attention output (via V) simultaneously. A single projection fault therefore perturbs scores and values at the same time. In decoders, K and V additionally enter the cache.

- (3) **M3: Residual bypass.** In $h = x + \text{MHA}(x)$, a perturbation δx appears in h with unit gain through the skip path. The skip connection therefore carries any change at x to the residual output even when the sublayer transformation reduces it, although subsequent normalization may reshape it.
- (4) **M4: Cross-layer propagation.** The residual stream provides repeated identity paths across stacked layers [79]. A change at layer ℓ can therefore reach later representations, although intervening normalization and nonlinear operations may reduce or reshape it.
- (5) **M5: Backward pass propagation.** A fault that changes the loss $\mathcal{L}$ can change $\partial\mathcal{L}/\partial\theta_i$ for parameters whose computation contributes to that loss term, which can affect weight updates across multiple components during training.
- (6) **M6: Architecture-wide intervention.** Some fault types inherently affect multiple components at once. A variant fault (e.g., single-head instead of multi-head attention) simultaneously changes the attention computation pattern, parameter shapes, and kernel dispatch. Unlike Mechanisms 1–5, this is not a propagation relation but a joint structural effect in which a single architectural change alters multiple components at the same time.
- (7) **M7: Cache propagation across time steps.** (decoder only). The KV cache stores K and V across generation steps. A cache fault affects the current token and all future tokens.

Three nonlinear operations limit propagation at specific edges. Softmax maps deviations into $[0, 1]$ with a sum-to-one constraint, which reduces large-magnitude shifts. LayerNorm [3] rescales outputs to zero mean and unit variance, which removes additive shifts. Activation functions (e.g., ReLU, GELU) suppress changes in their saturation regions and pass them in their linear regions. We annotate these limiting operations on the corresponding edges of the FPG. We can categorize the seven mechanisms fall into three classes (see Table 20 in Section Appendix D).

- *Propagation relations.* (M1, M2, M3, M5, M7) describe the structural paths through which a fault at one component can reach another (via forward data flow/residual identity/backward gradient flow/autoregressive state reuse). Message-passing graph uses the forward relations (M1, M2, M3, M7) as edges and represents M5 through gradient features.
- *Recursive composition.* (M4) is the repeated application of propagation relations 1 and 3 across stacked layers.
- *Structural intervention.* (M6) is a joint structural effect in which a single architectural change alters multiple components rather than propagating from one to another. Relation-typed message passing that distinguishes these three classes is a natural extension.

We derive the FPG from the transformer’s forward and backward computations, with forward data flow, residual paths, autoregressive state reuse, and gradient coupling each contributing distinct edges (Figure 6).

To use in the diagnostic model, we collapse the component-level FPG to a group-level adjacency matrix. Each transformer component maps to the feature group that measures it (Table 3). A group-level edge exists if any component in the source group has a propagation edge to any component in the target group. Components that map to the same group (e.g., attention masking, attention weights, and attention output all map to the Attention group) are merged. Structural groups receive FPG-derived neighbor edges. The cross-layer observation group (Representation drift) and model-wide context groups (Training dynamics, Validation performance) participate through self-loops only. They receive the same transformation as structural groups, but do not receive information from neighboring groups. Groups that map to multiple components inherit all edges of their constituent components. For the decoder diagnostic model ($G = 13$), the near-diagonal sparsity of the group-level adjacency matrix $\hat{A}$ reflects the sequential data flow through the transformer block (see Figure 7). The off-diagonal entries (e.g., QKV→Cache, Cache→Attention, FFN→Residual) capture the feedback and branching paths that the FPG encodes. Non-structural groups (Representation drift, Training dynamics, Validation performance) appear as isolated self-loops in the lower-right block. We construct $\hat{A}$ from the

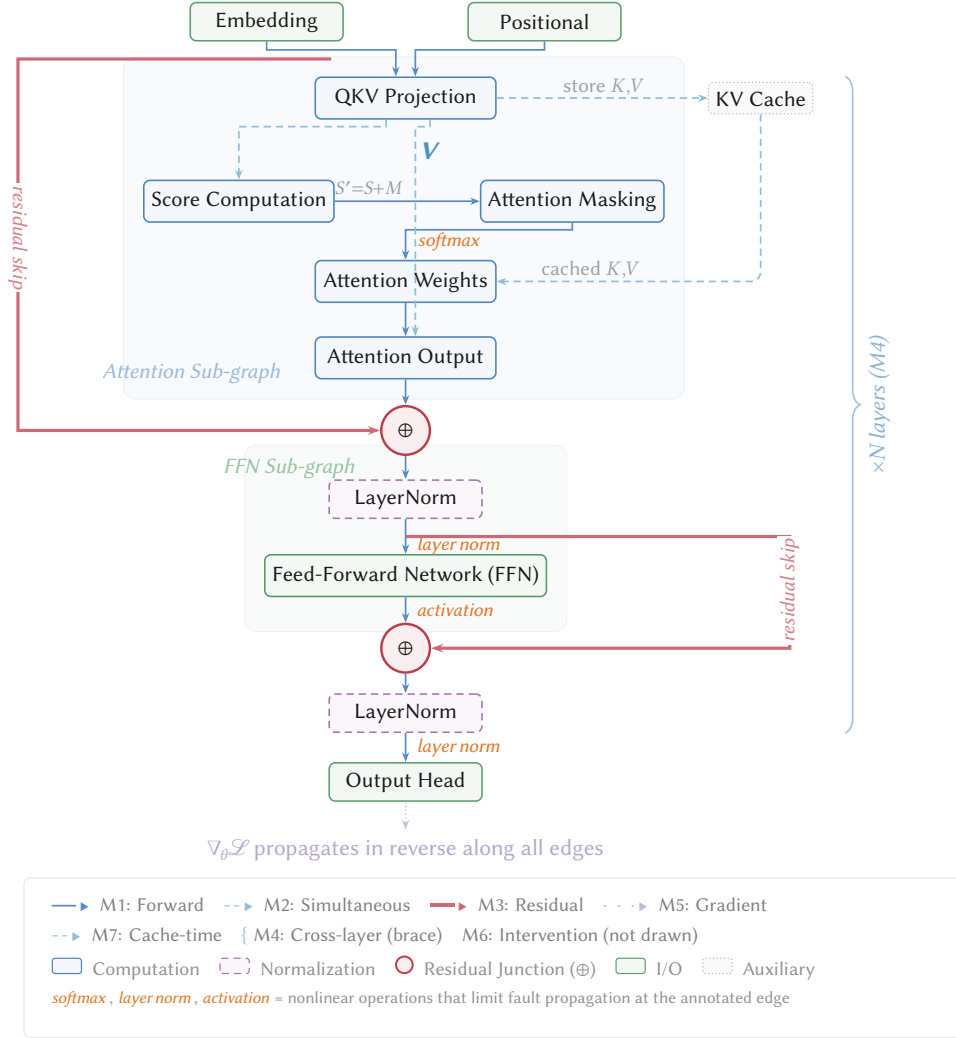


Fig. 6. Fault Propagation Graph (FPG) for transformer architectures

forward and structural mechanisms (M1, M2, M3, M4, M7), capturing the transformer's forward data-flow graph at the group level. Backward gradient coupling (M5) enters through the gradient features within each group (Section 4.2) and the architecture-wide intervention (M6) enters through the fault labels rather than through edges (Table 20). The near-diagonal sparsity of $\hat{\mathbf{A}}$ in Figure 7 represents this forward data-flow structure.

4.2 Feature Representation

The Fault Propagation Graph defines which transformer components can affect each other, but the diagnostic model requires quantitative measurements of those components during training. Generic DNN fault features such as loss trajectories, gradient statistics, and activation counts do not distinguish transformer fault categories. Corrupted QKV projections, missing causal masks, and misconfigured positional encodings leave end-task metrics untouched but produce

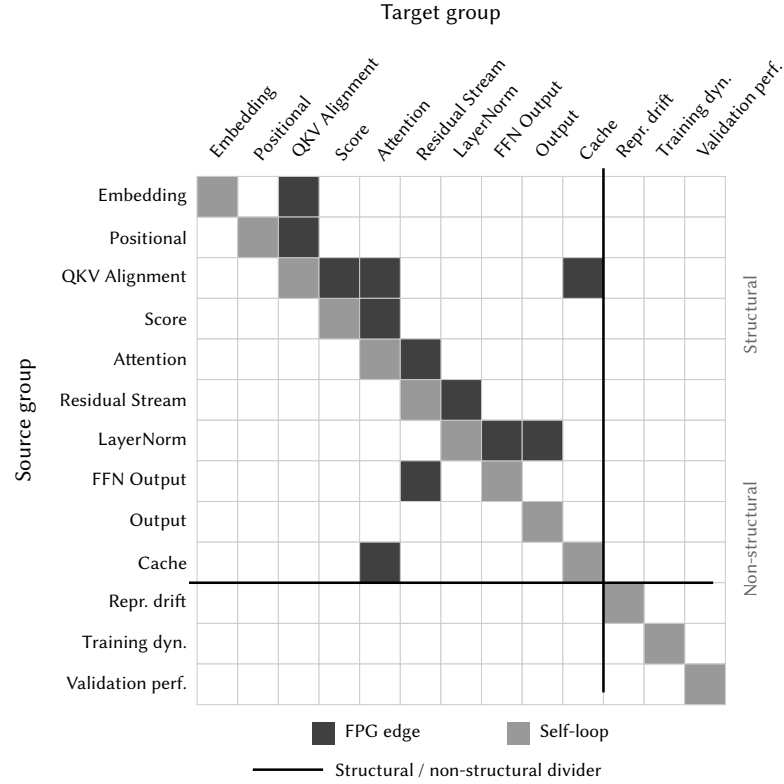
Fig. 7. Group-level adjacency matrix $\hat{A}$ for the decoder diagnostic model

Table 3. Feature-group roles and message passing in DEFault++

Feature Group	Role	Scope	Message Passing
Attention	Structural	Attention masking, weights, output	FPG edges
Score	Structural	Score computation	FPG edges
FFN output	Structural	FFN	FPG edges
LayerNorm	Structural	LayerNorm	FPG edges
Residual stream	Structural	Residual connections	FPG edges
QKV alignment	Structural	QKV projection	FPG edges
Embedding	Structural	Embedding	FPG edges
Positional	Structural	Positional encoding	FPG edges
Output	Structural	Output head	FPG edges
Cache (decoder)	Structural	KV cache	FPG edges
Representation drift	Cross-layer observation	Inter-layer residual path	Self-loop only
Training dynamics	Model-wide context	Model-wide optimization	Self-loop only
Validation performance	Model-wide context	Model-wide output quality	Self-loop only

characteristic patterns in attention-level and component-level measurements. We therefore define a transformer-specific feature space that targets attention distributions, projection alignments, and residual-stream behavior (see Figure 8 for the feature-construction process and Table 4 for the feature groups, their metric types, and their collection branches).

The diagnostic model uses 13 feature groups for decoders and 12 for encoders. The Cache group applies only to decoders, so the encoder count is the 13 groups in Table 4 minus the Cache group. The metric-type partition C_{int} , C_{opt} , C_{train} , C_{eval} is separate from the feature groups. The partition sorts metrics by when they are collected (per layer, per training step, per epoch) rather than by feature group. Every metric still belongs to one of the 13 feature groups. Each metric definition states the group assignment for its metric type. The gradient metrics in particular are appended to the feature group of their component rather than forming a group of their own.

Table 4. Feature groups, metric types, and feature dimensions in DEFault++

Feature Group	Metric Types	n_g
<i>Layer-internal metrics (C_{int}, collected at each training step, per layer)</i>		
Attention	Attention entropy, padding attention mass, inter-head cosine similarity, head utilization, attention rank, 6/7 ^a	
Score	cross-example leakage, future attention mass (decoder only)	
Score	Pre-softmax score	1
FFN output	FFN output norm	1
LayerNorm	Scale parameter norm, post-norm distribution	2
Residual stream	Residual cosine similarity	1
Repr. drift	CKA layer similarity	1
QKV alignment	Q-K sim., Q-V sim., K-V sim.	3
<i>Gradient metrics (C_{opt}, collected at each training step)</i>		
Optimization	Gradient and update features (gradient norm, update ratio, update activity)	21
<i>Behavioral metrics (C_{train}, collected at each training step)</i>		
Embedding	Embedding norm, token-level variance	2
Positional	Positional sensitivity	1
Training dyn.	Loss trajectory, gradient norm volatility, step time, peak memory allocation	4
Output	Prediction confidence, output entropy, margin statistics	3
Cache	Cache hidden similarity, cache distribution divergence	2
<i>Validation metrics (C_{eval}, collected at epoch end or validation checkpoints)</i>		
Validation perf.	Task accuracy / perplexity, calibration error	2

^a6 for encoders, 7 for decoders (future attention mass applies only to decoders).

A. LAYER-INTERNAL METRICS (C_{int} , collected per layer at each training step).

Each metric type yields one scalar per layer before aggregation. When a metric is defined as a distribution or a set of moments, we reduce it to a single scalar at extraction time, and the definition states which scalar. The integer counts in Table 4 are therefore scalar counts per layer.

A1-Attention. We measure seven properties of the post-softmax attention distribution at each sampled layer (six for encoders, seven for decoders).

- Attention entropy (H_{attn}) computes the Shannon entropy of the attention distribution, averaged across heads and query positions [90]. Low entropy indicates that a head concentrates on a single position. High entropy indicates a near-uniform distribution.

- Padding attention mass (PadMass) computes the fraction of attention directed at padding tokens. Non-zero padding mass indicates a masking fault.
- Inter-head cosine similarity (HeadSim) computes pairwise cosine similarity between the flattened attention patterns of heads i and j within each layer. High pairwise similarity indicates redundant heads.

$$\text{sim}(\mathbf{a}_i, \mathbf{a}_j) = \frac{\mathbf{a}_i \cdot \mathbf{a}_j}{\|\mathbf{a}_i\| \|\mathbf{a}_j\|} \quad (8)$$

- Head utilization is the fraction of heads whose attention entropy exceeds a minimum threshold, which indicates that the head attends to more than one position. The fraction is one scalar per layer, and the replication package [36] reports the threshold value. Variant faults that reduce the effective head count lower this fraction [11, 90].
- Attention rank quantifies the effective dimensionality of each head's attention weight matrix $\mathbf{A}_i \in \mathbb{R}^{n_q \times n_k}$ as the exponential of the Shannon entropy of its normalized singular value distribution [80].

$$\text{EffRank}(\mathbf{A}_i) = \exp\left(-\sum_{k=1}^n \tilde{\sigma}_k \log \tilde{\sigma}_k\right); \quad \tilde{\sigma}_k = \frac{\sigma_k}{\sum_{j=1}^n \sigma_j} \quad (9)$$

where $n = \min(n_q, n_k)$ is the number of singular values of $\mathbf{A}_i$, $\tilde{\sigma}_k$ is the k -th singular value normalized to sum to one, and n_q and n_k are the numbers of query and key positions. For self-attention within a single sequence, $n = n_q = n_k$. A rank near 1 indicates a low-dimensional attention pattern dominated by a single singular component. A rank approaching n indicates a broader singular value spectrum across query positions. We average the effective rank across heads to produce one scalar per layer. Variant faults that reduce effective multi-head capacity lower the metric, while faults that introduce more diffuse high-dimensional structure may raise it.

- Cross-example leakage measures the total attention weight flowing between unrelated examples in the same batch, averaged over heads and layers [13, 89]. Under correct masking, this quantity is zero. Non-zero values indicate that the attention mechanism attends across example boundaries, which is a direct indicator of masking faults.
- Future attention mass (decoder only) measures the total attention weight assigned from each query position to future key positions, averaged over heads and layers [89]. Under correct causal masking, this quantity is near zero, and causal-mask violations increase it systematically.

A2-Score. We record the raw attention logits before the softmax operation [89] as follows.

$$s = QK^\top / \sqrt{d_k} \quad (10)$$

At each layer, we reduce the full query-key score matrix to a single scalar by computing its mean across all positions [16]. This within-layer reduction precedes the cross-layer aggregation (A_ℓ) described above. High score variance across layers indicates that attention sharpness differs substantially between early and late layers, which can result from softmax saturation and reduced gradient flow [16, 105]. Pre-softmax scores are sensitive to Score and Positional faults because both fault types directly alter this computation.

A3-FFN output. We record the ℓ_2 norm of the feed-forward sublayer outputs at each layer, computed as $\|\text{FFN}(x)\|_2$ per token and averaged across the batch. Abnormally large norms indicate instability in the FFN transformation. Near-zero norms indicate that the FFN sublayer contributes little to the residual stream, which can occur when activation functions suppress most dimensions.

A4-LayerNorm. We record two scalars per layer for layer normalization [3]. Frobenius norm $\|y\|_F$ of the learned scale vector and the standard deviation of the post-norm output $\hat{x}$ across the hidden dimension.

$$\hat{x} = y \odot \frac{x - \mu}{\sigma + \epsilon} + \beta \quad (11)$$

Under correct layer normalization, the term $(x - \mu)/(\sigma + \epsilon)$ has unit variance per token before the scale y and bias β are applied. The post-norm standard deviation therefore tracks $\|y\|$ in expectation. A LayerNorm fault that corrupts the scale parameter, the bias parameter, or the stability term ϵ moves the post-norm standard deviation or the y magnitude outside the range seen in clean training runs.

A5-Residual stream. We compute the cosine similarity between the input and output of each residual sublayer,

$$\text{res_sim}_\ell = \frac{x_\ell \cdot h_\ell}{\|x_\ell\| \|h_\ell\|} \quad (12)$$

where x_ℓ is the sublayer input and $h_\ell = x_\ell + f(x_\ell)$ is the residual output at layer ℓ . In a correctly functioning transformer, the skip connection preserves the input direction while the sublayer f adds a controlled change, which keeps cosine similarity close to 1.0. A Residual fault that corrupts the skip-connection path reduces this similarity. This metric measures the effect of a fault *within* a single sublayer (input vs. output of the same residual block).

A6-Representation drift. We compute Centered Kernel Alignment (CKA) [46] between consecutive layers,

$$\text{CKA}(H_\ell, H_{\ell+1}) = \frac{\|H_{\ell+1}^\top H_\ell\|_F^2}{\|H_\ell^\top H_\ell\|_F \|H_{\ell+1}^\top H_{\ell+1}\|_F} \quad (13)$$

where $H_\ell \in \mathbb{R}^{n \times d}$ is the centered representation matrix at layer ℓ . CKA measures similarity independent of rotation and isotropic scaling. A fault that propagates across multiple layers causes decreasing CKA between adjacent layers. We use CKA rather than direct cosine similarity because CKA is invariant to orthogonal transformations of the representation space. Unlike the Residual stream metric, which measures how a sublayer's output differs from its input, CKA measures cumulative representational change *between* consecutive layers and therefore captures faults that propagate beyond the originating sublayer.

A7-QKV alignment. We compute pairwise cosine similarity between the query, key, and value projection outputs at each sampled layer:

$$\text{sim}(Q, K) = \frac{Q \cdot K}{\|Q\| \|K\|}, \quad \text{sim}(Q, V) = \frac{Q \cdot V}{\|Q\| \|V\|}, \quad \text{sim}(K, V) = \frac{K \cdot V}{\|K\| \|V\|} \quad (14)$$

where $Q = W_Q x$, $K = W_K x$, and $V = W_V x$ are the projection outputs for input x [89]. We compute each similarity per token and average it across the batch and across non-padding token positions, which yields three scalars per layer. A fault that corrupts one projection weight matrix shifts its output direction relative to the other two, producing a measurable drop in pairwise similarity.

B. GRADIENT METRICS (C_{opt} , collected per component at each training step).

For each parameterized component (e.g., Attention, QKV, FFN, LayerNorm, Embedding), we compute three gradient-level measurements inspired by prior DNN fault detection work [7, 39]. *Gradient norm* computes the ℓ_2 norm of the gradient vector for all parameters belonging to that component:

$$g_c = \sqrt{\sum_{\theta \in \Theta_c} \|\nabla_\theta \mathcal{L}\|^2} \quad (15)$$

where Θ_c is the parameter set of component c . *Update ratio* measures the relative magnitude of weight change between consecutive training steps:

$$u_c = \frac{\|W_c^{(t)} - W_c^{(t-1)}\|_F}{\|W_c^{(t-1)}\|_F + \epsilon} \quad (16)$$

where $W_c^{(t)}$ is the weight matrix of component c at step t . *Update activity* is a binary flag indicating whether the gradient norm exceeds a minimum threshold ($g_c > 10^{-6}$), which identifies components that have stopped learning. These 15 component-level measurements (three per component) together with six global gradient statistics form the $C_{opt} = 21$ gradient metrics in Table 4. The Output group uses behavioral metrics only and does not add a sixth component-routed gradient set. Moreover, in the Fault Propagation Graph (FPG), each gradient feature computed at the component level is added to the feature set of its respective module (e.g., Attention, QKV, Cache). In addition, we include six global gradient statistics in the Training dynamics group. As a result, the message-passing graph comprises 12 groups for encoders and 13 for decoders (Table 3).

C. BEHAVIORAL METRICS (C_{train} , collected at each training step).

C1-Embedding. We record two properties of the input embedding vectors. *Embedding norm* is the mean ℓ_2 norm $\frac{1}{n} \sum_{i=1}^n \|e_i\|_2$ across all n input tokens, where $e_i = E[x_i]$ is the embedding of token x_i . *Token-level variance* is $\text{Var}(\{e_1, \dots, e_n\})$ computed across the hidden dimension. An Embedding fault that corrupts the embedding matrix E produces abnormal norms or collapses the variance across tokens. We compute both metrics on the first hidden state before the first transformer block.

C2-Positional. We measure positional sensitivity as the model’s response to a fixed non-zero shift in position indices on a held-out batch. In all experiments, we set $\Delta = 1$. Let $P_{\max}$ denote the model’s maximum supported position index, and let $\mathcal{I}(X)$ be the set of valid non-padding token positions in batch X such that $i + \Delta < P_{\max}$. For each $i \in \mathcal{I}(X)$, we compare the hidden state at position i under the original indices with the hidden state at the same token position under the shifted indices. We define positional sensitivity as,

$$S_{\text{pos}}(X) = \frac{1}{L |\mathcal{I}(X)|} \sum_{\ell=1}^L \sum_{i \in \mathcal{I}(X)} \|h_{\ell,i}(X) - h_{\ell,i}^{(\Delta)}(X)\|_2 \quad (17)$$

where L is the number of instrumented layers. Tokens that would exceed the valid index range after shifting are excluded from the average. We do not clip, wrap, or reuse position indices. Larger values indicate stronger dependence of the model’s representations on positional information. Positional faults that omit, misalign, or truncate positional encoding shift this quantity away from the clean baseline.

C3-Training dynamics. We record four optimization-level and infrastructure-level metrics at each training step. *Loss trajectory* is the scalar training loss per step. *Gradient norm volatility* measures how much the gradient magnitude varies over a rolling window of 20 training steps, computed as the squared coefficient of variation of the gradient norm.

$$\text{GNV} = \frac{\text{Var}_t(\|\nabla \mathcal{L}_t\|)}{\mathbb{E}_t[\|\nabla \mathcal{L}_t\|]^2} \quad (18)$$

Here $\nabla \mathcal{L}_t$ is the full-batch gradient at training step t , and the variance and expectation are taken across the steps in the window. High values indicate that the gradient magnitude is unstable across consecutive optimizer steps, which can result from a learning-rate misconfiguration, optimizer-state corruption, or interaction with mixed-precision arithmetic. We use this temporal-variance form because our training logs record per-step full-batch gradients rather than the

per-microbatch gradients required to compute the per-example gradient noise scale of McCandlish et al. [60]. We treat the temporal form as a related and less data-intensive proxy for gradient stability. *Step time* is the wall-clock duration of each training step. Sudden increases in step time can indicate memory pressure or scheduling faults. *Peak memory allocation* records the peak GPU memory reported by the CUDA allocator during each training step [9, 69]. A forced fallback to a non-optimized kernel or an unintended data-layout change can increase peak memory without raising an explicit error [13]. We include peak memory alongside step time because both are infrastructure-level measurements that detect kernel and scheduling faults.

C4-Output. We record three properties of the output head at each training step. *Prediction confidence* computes the mean maximum softmax probability $\frac{1}{N} \sum_{i=1}^N \max_c p_c^{(i)}$ across all samples in the batch. *Output entropy* computes the mean Shannon entropy of the output distribution $-\frac{1}{N} \sum_{i=1}^N \sum_c p_c^{(i)} \log p_c^{(i)}$. *Margin statistics* measure the gap between the correct-class logit and the strongest competing logit. For encoder classification tasks, we compute these metrics over the classification logits per batch sample. For decoder language modeling tasks, we average over non-padding tokens within each sequence, where margin measures the gap between the logit of the true next token and the highest competing logit. Low confidence, high entropy, or negative margins indicate that the output head does not produce well-separated predictions, whether because of upstream faults or direct output-projection faults.

C5-Cache (decoder only). We measure two properties of the key-value cache used during autoregressive generation. *Cache hidden similarity* computes the cosine similarity $\text{sim}(K_{\text{cached}}, K_{\text{recomputed}})$ between cached and recomputed key states at each generation step. High similarity indicates a consistent cache. Low similarity indicates stale or corrupted cached states. *Cache distribution divergence* computes the KL divergence as follows.

$$D_{\text{KL}}(P_{\text{cache}} \| P_{\text{fresh}}) = \sum_v P_{\text{cache}}(v) \log \frac{P_{\text{cache}}(v)}{P_{\text{fresh}}(v)}, \quad (19)$$

between the next-token distribution produced using the cache (P_{cache}) and the distribution produced by fresh computation (P_{fresh}). Non-zero divergence indicates that cached states produce different predictions, which is a direct indicator of KV Cache faults.

D. VALIDATION METRICS (C_{eval} , collected at epoch end or validation checkpoints).

We record two measurements at epoch end or at scheduled validation checkpoints. *Task accuracy* is classification accuracy for encoder tasks and perplexity for decoder language modeling tasks. *Calibration error* is [26],

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{N} |\text{acc}(B_b) - \text{conf}(B_b)|, \quad (20)$$

where B_b is the set of samples whose maximum predicted probability falls in bin b , $|B_b|$ is the size of bin b , N is the total number of samples, $\text{acc}(B_b)$ is the fraction of samples in B_b that are correctly classified, and $\text{conf}(B_b)$ is the average maximum predicted probability across samples in B_b . We use $B = 10$ equal-width confidence bins. Faults that degrade downstream performance produce abnormal values in these metrics. Faults that shift internal behavior without affecting validation performance produce normal values in this group but abnormal values in other groups. Since these metrics are computed at epoch boundaries rather than at each training step, they enter the feature-construction process after the epoch-level summary stage (Table 4).

4.2.1 Aggregation & Feature Vector Construction. We classify each metric into one of four collection branches based on its granularity (see Figure 8). Prior work has shown that attention distributions [11, 90], representation geometry [20, 46], normalization parameters [3, 103], and residual propagation [79] vary qualitatively across layers. We therefore measure

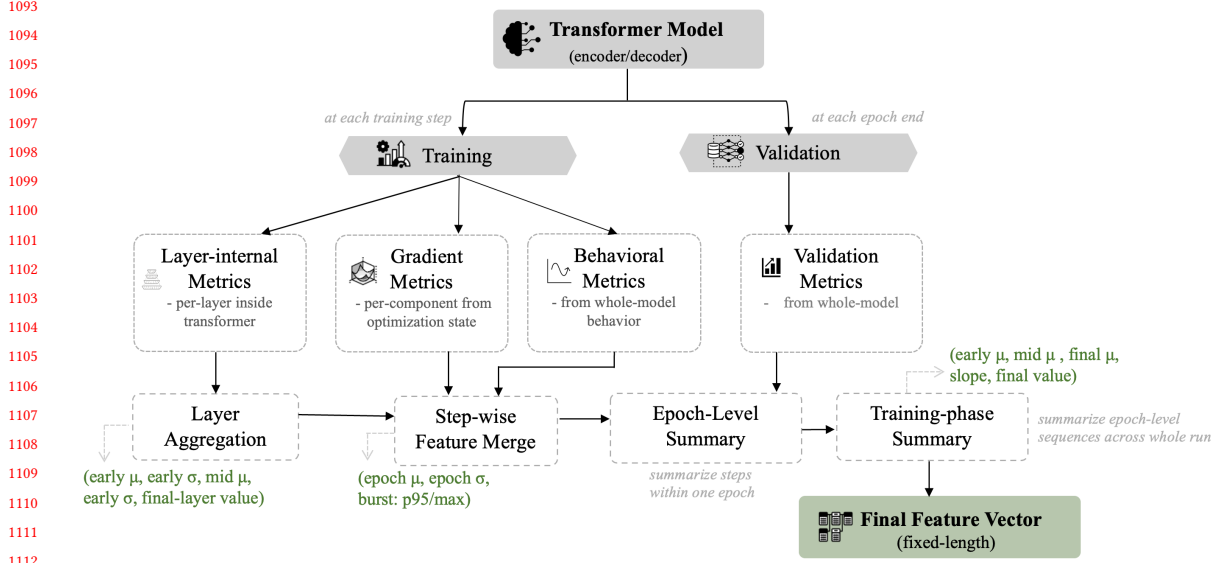


Fig. 8. DEfault++ feature construction, from raw measurements to the fixed-length feature vector

$C_{int} = 15$ (encoder) or $C_{int} = 16$ (decoder) layer-internal metric types at each layer independently (Table 4). The decoder count is higher because future attention mass applies only to decoder architectures. The remaining metrics are collected at the step level or epoch level. Gradient metrics ($C_{opt} = 21$) record gradient norm, update ratio, and update activity across parameterized components and global gradients. Behavioral metrics ($C_{train} = 10$ for encoders, 12 for decoders) measure model-wide properties such as training loss, output-head behavior, positional sensitivity under controlled position shifts, and peak memory allocation. Validation metrics ($C_{eval} = 2$) record accuracy or perplexity and calibration error at epoch boundaries. To create a fixed-length representation regardless of model depth, we aggregate per-layer layer-internal metrics into five summary statistics ($A_\ell = 5$: early mean, early standard deviation, mid mean, mid standard deviation, and final-layer value). A 6-layer and a 12-layer model therefore produce the same number of features per group. Step-level metrics (gradient and behavioral) skip this aggregation. After layer aggregation produces $d_{step} = A_\ell C_{int} + C_{opt} + C_{train}$ step-level features, we apply two further aggregation stages to convert variable-length training runs into a fixed-length vector.

- *Epoch-level summary* ($A_e = 3$). Following prior work on step-level training metrics [4, 96], we aggregate step-level values within each training epoch into three statistics: epoch mean, epoch standard deviation, and a burst statistic (the 95th-percentile or maximum value). These statistics capture micro-level training behavior within a single pass over the data. Abnormal within-epoch variance can reveal batch-level issues such as gradient instability [4] or data-complexity mismatches [96] that disappear when only the final epoch value is recorded. Validation metrics ($C_{eval} = 2$) enter at this stage because they are collected at epoch end or at validation checkpoints. The epoch-level vector has dimensionality $d_{epoch} = A_e d_{step} + C_{eval}$.

- *Training-phase summary* ($A_p = 5$). We partition the epoch sequence into three training phases (early, mid, final) and compute the mean within each phase, a linear-regression slope across phases, and the final-epoch value. Together these five statistics per metric compose $A_p = 5$. The slope captures the macro-level trajectory of the training process. A fault that causes learning-rate decay to stall [107], or that traps the optimizer in a suboptimal region [96], produces a

characteristic slope pattern that a single-epoch snapshot would miss. Final values capture faults whose effect is visible only at convergence, independent of the trajectory shape.

- *Feature vector length.* Layer-internal metrics (C_{int}) pass through layer aggregation ($A_\ell = 5$) before the epoch-level and training-phase summaries. Gradient and behavioral metrics skip layer aggregation. Validation metrics enter after the epoch-level summary. The raw feature vector length before filtering is,

$$d_{\text{final}} = A_p (A_\ell (C_{int} + C_{opt} + C_{train}) + C_{eval}) \quad (21)$$

For encoders, $C_{int} = 15$, $C_{opt} = 21$, $C_{train} = 10$, and $C_{eval} = 2$.

$$d_{\text{final}} = 5 \times (3 \times (5 \times 15 + 21 + 10) + 2) = 1600$$

For decoders, $C_{int} = 16$ as decoder features add future attention mass and $C_{train} = 12$ for two additional cache metrics.

$$d_{\text{final}} = 5 \times (3 \times (5 \times 16 + 21 + 12) + 2) = 1705$$

We apply coefficient-of-variation filtering ($CV < 0.01$) to remove near-constant columns. The fixed-length feature vector serves as input to the diagnostic model (Section 4.3).

4.3 Diagnostic Model

DEFault++ performs hierarchical diagnosis over a single input instance (see Algorithm 1). At each level, the model either makes a prediction or passes the instance to the next level, while the final level also generates an explanation that provides additional insight into the root cause.

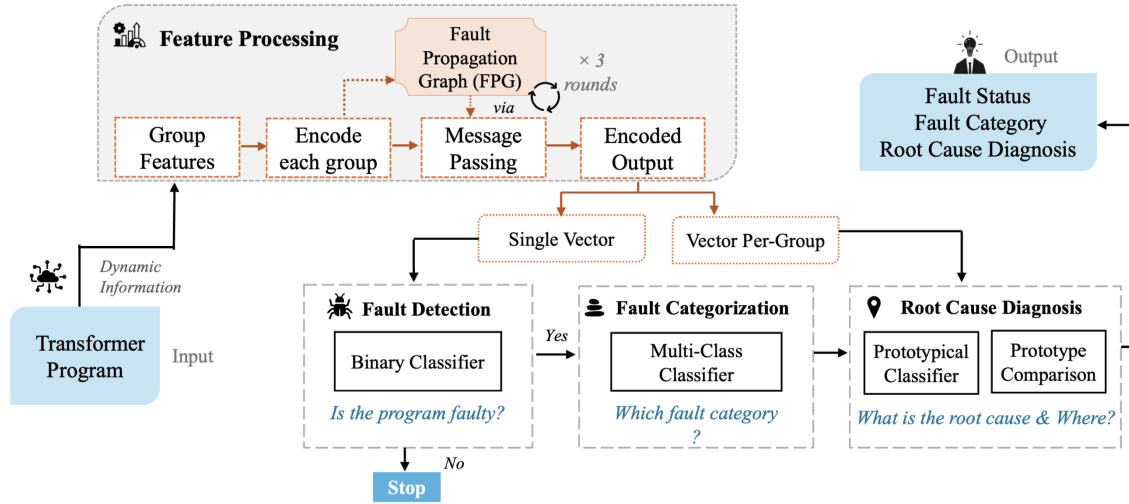


Fig. 9. Overview of DEFault++ for fault detection, categorization, and root-cause diagnosis

4.3.1 Feature-Group Encoding. We encode each feature group with a dedicated MLP. We keep groups structurally separate so that FPG message passing can mix information across them and the explanation step can examine each group's embedding independently. We use trainable MLPs rather than a flat classifier because the prototype-distance objective and the contrastive separation loss both require gradients to flow through the group-level weights during

training. As encoder and decoder architectures differ in the number of feature groups ($G = 12$ for encoders and $G = 13$ for decoders, with the Cache group decoder-only as noted at the start of Section 4.2), we train separate encoder and decoder diagnostic models. Given the feature vector $\mathbf{z} \in \mathbb{R}^d$ partitioned into G groups $\{\mathbf{z}_{g_1}, \dots, \mathbf{z}_{g_G}\}$ (see Table 4), each group g is encoded by a dedicated MLP (Equation (22)).

$$\mathbf{h}_g = \text{MLP}_g(\mathbf{z}_g) \in \mathbb{R}^h, \quad g = 1, \dots, G, \quad (22)$$

where $h = 32$ is the hidden dimension per group (selected via grid search on the inner validation fold). The group embeddings form the matrix $\mathbf{H} = [\mathbf{h}_1; \dots; \mathbf{h}_G] \in \mathbb{R}^{G \times h}$. A projection layer maps the concatenated group embeddings to a shared representation (Equation (23)).

$$\mathbf{z}_{\text{proj}} = W_{\text{proj}} \text{vec}(\mathbf{H}) \in \mathbb{R}^e, \quad (23)$$

where $e = 64$ is the embedding dimension and $\text{vec}(\mathbf{H}) \in \mathbb{R}^{Gh}$ denotes the row-major vectorization. Levels 1 and 2 use the pooled projection $\mathbf{z}_{\text{proj}}$ for detection and categorization. Level 3 uses the group embedding matrix $\mathbf{H}$ directly to preserve group-level structure for root-cause separation and group-level explanation (Section 4.5).

4.3.2 Message Passing in the Fault Propagation Graph. We update each group embedding by aggregating information from its neighbors in the FPG. One round of message passing replaces a group’s embedding with a learned mixture of its own current value and the embeddings of the groups linked to it by FPG edges. After several rounds, each group embedding reflects evidence from the groups within a few hops of it along the FPG. The QKV alignment embedding, for example, then reflects not only QKV measurements but also their effect on attention scores and the residual stream, the components a QKV projection fault reaches through Mechanisms 1 and 2. We construct the group-level adjacency matrix $\mathbf{A} \in \mathbb{R}^{G \times G}$ from the component-level FPG (Section 4.1) and the mapping in Table 3. Structural groups receive FPG-derived neighbor edges. The cross-layer observation group (Representation drift) and the model-wide context groups (Training dynamics, Validation performance) receive self-loops only, so they undergo the learned transformation but do not aggregate from neighbors. Following graph convolutional networks [44], one round of message passing is

$$\mathbf{H}' = \text{ReLU}(\hat{\mathbf{A}} \mathbf{H} W_{\text{msg}}), \quad (24)$$

where $\hat{\mathbf{A}}$ is the row-normalized adjacency matrix with self-loops and $W_{\text{msg}} \in \mathbb{R}^{h \times h}$ is a learnable weight matrix. We use row normalization rather than symmetric normalization so that each group aggregates its neighbors at a comparable scale regardless of degree. We compared the two schemes on the inner validation fold and adopted row normalization for its more stable training curves with no loss in held-out Macro-F1. We apply three rounds of message passing, each with its own W_{msg} . We selected three rounds through grid search on the inner validation fold over the set $\{1, 2, 3, 4, 5\}$. Three rounds extend the receptive field of each group embedding to its three-hop neighborhood in the group-level adjacency. Three rounds are enough to cover the short-range propagation patterns that matter most for categorization, such as QKV $\rightarrow$ Score $\rightarrow$ Attention $\rightarrow$ Residual, while staying below the depth at which graph convolutional networks begin to oversmooth [48]. Three rounds do not span the full diameter of the group-level graph. Groups separated by more than three hops therefore exchange information through repeated short-range aggregation rather than through a single long-range step. The updated embeddings $\mathbf{H}'$ replace $\mathbf{H}$ in both the projection and the Level 3 prototype matching.

4.3.3 Hierarchical Classification. All three classification levels share one encoder (group encoding, FPG message passing, and projection) rather than one per level. Sharing reduces parameters and training cost. Message passing over a 12–13-node graph with 32-dimensional embeddings is negligible relative to the upstream transformer training. The

FPG also encodes structural dependencies between transformer components. A fault in one component (e.g., QKV projection) can shift measurements in downstream components (e.g., attention scores, residual stream). Message passing aggregates these cross-component dependencies into each group embedding, which benefits all three levels to varying degrees. Each level adds a task-specific head.

- *Level 1: Fault detection.* A two-layer MLP maps $\mathbf{z}_{\text{proj}}$ to binary logits (faulty vs. correct).
- *Level 2: Fault categorization.* A two-layer MLP maps $\mathbf{z}_{\text{proj}}$ to C -class logits, where $C = 11$ for encoders and $C = 12$ for decoders. During training, the class-weighted cross-entropy is computed only over samples labeled faulty.
- *Level 3: Root-cause diagnosis.* Level 3 separates the 2–5 root causes within a fault category. Per-root-cause training support ranges from fewer than 10 samples (e.g., Variant encoder) to approximately 80, with an average of 42 per root cause for encoders and 26 for decoders. A standard softmax head over this label space has two weaknesses under our data budget. The head overfits to majority root causes when per-class support is small, and it returns a single prediction score that cannot be attributed back to individual feature groups, which the explanation step needs.

We therefore pair a per-category cross-entropy head with a prototypical classifier [41, 85]. A prototypical classifier represents each root cause by a prototype, the centroid of its training examples in the embedding space, and assigns a new sample to the nearest prototype. Two properties make it the right fit here. The prototype is a simple average rather than a learned weight vector, which is stable when per-class support is small, and the distance to a prototype decomposes group by group, which gives the explanation step in Section 4.5 a per-group contribution score. The cross-entropy head supplies gradient signal to the shared encoder during training, while the inference-time decision uses prototypes only.

First, a per-category classification head maps $\mathbf{z}_{\text{proj}}$ to root-cause logits within each category. During training, the cross-entropy loss is computed only over faulty samples whose ground-truth fault category matches the category under training. Negative samples for Level 3 are the other root causes within the same category, not samples from different categories. Second, a prototypical classifier [50, 85] operates in the group embedding space $\mathbf{H} \in \mathbb{R}^{G \times h}$. Distances in $\mathbf{H}$ are converted to class logits through the prototype-matching loss (Equation (30)). Let $\mathcal{D}_{c,r}$ denote the set of training samples with category c and root cause r . For each root cause r within category c , we compute a prototype $\boldsymbol{\pi}_{c,r}$ as the mean group embedding (Equation (25)).

$$\boldsymbol{\pi}_{c,r} = \frac{1}{|\mathcal{D}_{c,r}|} \sum_{i \in \mathcal{D}_{c,r}} \mathbf{H}_i \in \mathbb{R}^{G \times h}. \quad (25)$$

The distance from a sample to a prototype decomposes additively by group (Equation (26)).

$$d(\mathbf{H}, \boldsymbol{\pi}_{c,r}) = \sum_{g=1}^G \|\mathbf{h}_g - \boldsymbol{\pi}_{c,r,g}\|^2 = \sum_{g=1}^G d_g, \quad (26)$$

where $d_g = \|\mathbf{h}_g - \boldsymbol{\pi}_{c,r,g}\|^2$ is the squared distance contributed by group g . Level 3 predicts the root cause whose prototype is nearest: $\hat{r} = \arg \min_{r \in \mathcal{R}_c} d(\mathbf{H}, \boldsymbol{\pi}_{c,r})$. At inference, Level 3 runs only after Level 2 predicts a fault category, and root-cause diagnosis is restricted to that category. The per-category cross-entropy heads supply auxiliary gradient information to the shared encoder during training, and the prediction itself uses the prototype classifier, whose distance decomposes additively by group (Equation (26)) and supports the group-level explanation in Section 4.5.

Algorithm 1 DEFault++ hierarchical diagnosis for one input instance

Input: Feature vector $\mathbf{x}$; shared encoder (group embeddings $\{\mathbf{h}_g\}_{g=1}^G$ and projection $\mathbf{z}_{\text{proj}}$); trained prototypes $\{\boldsymbol{\pi}_{c,r}\}$

Output: Detection label $\hat{y}$; if faulty: fault category $\hat{c}$, root cause $\hat{r}$, and per-group importance scores $\{w_g\}_{g=1}^G$

► *Level 1: Fault Detection*

- 1: Classify the instance as faulty or correct: $\hat{y} \leftarrow \text{Detect}(\mathbf{z}_{\text{proj}})$
- 2: **if** $\hat{y} = \text{correct}$ **then**
- 3: **return** $\hat{y}$ ► correct instances exit here;
- 4: **end if**
- *Level 2: Fault Categorization*
- 5: Assign the fault to a transformer component category: $\hat{c} \leftarrow \text{Categorize}(\mathbf{z}_{\text{proj}})$
- *Level 3: Root-Cause Diagnosis*
- 6: **for** each root cause r in the predicted category $\mathcal{R}_{\hat{c}}$ **do**
- 7: Sum squared distances to prototype r across all feature groups: $d_r \leftarrow \sum_{g=1}^G \|\mathbf{h}_g - \boldsymbol{\pi}_{\hat{c},r,g}\|^2$
- 8: **end for**
- 9: Select the nearest prototype as the predicted root cause: $\hat{r} \leftarrow \arg \min_{r \in \mathcal{R}_{\hat{c}}} d_r$
- *Level 3: Root-Cause Explanation*
- 10: Identify the nearest alternative root cause: $\tilde{r} \leftarrow \arg \min_{r \in \mathcal{R}_{\hat{c}} \setminus \{\hat{r}\}} d_r$
- 11: **for** each feature group $g = 1, \dots, G$ **do**
- 12: Compute how much group g favors $\hat{r}$ over $\tilde{r}$: $\Delta_g \leftarrow d_g(\boldsymbol{\pi}_{\hat{c},\tilde{r}}) - d_g(\boldsymbol{\pi}_{\hat{c},\hat{r}})$
- 13: Normalize positive contributions into importance score: $w_g \leftarrow \max(\Delta_g, 0) / \sum_{g'} \max(\Delta_{g'}, 0)$
- 14: **end for**
- 15: **return** $\hat{y}, \hat{c}, \hat{r}, \{w_g\}_{g=1}^G$

4.4 Training Objective

We design the training objective with four components. Two components correspond to conventional cross-entropy loss terms for fault detection and fault categorization, respectively. The remaining two components are specifically designed to facilitate Level 3 root-cause separation. In Level 3, we introduce a novel training formulation for fault diagnosis that combines supervised contrastive learning [42] with prototype matching [85]. The contrastive part encourages samples with the same root cause to stay close in the learned space and samples with different root causes to stay apart. The prototype-matching part then compares each sample with the learned prototype of each root cause and assigns the closest match. This formulation differs from prior DL fault detection and diagnosis techniques by using both root-cause separation and prototype-based matching in the same training objective. We test the effect of the separation loss in RQ3.

At inference time, DEFault++ runs three levels hierarchically. Level 1 filters instances before passing them to Level 2, and Level 2 filters instances before passing them to Level 3. During training, the three levels share a single encoder through hard parameter sharing [8, 81], and we optimize them jointly. The dataset contains only 5,556 instances, and several root causes have limited sample counts (Section 3). Training a separate encoder for each level would split this limited supervision across multiple models. We therefore train all three levels jointly with a shared encoder, since the tasks are hierarchically nested. Each Level 3 sample also belongs to Level 2 and Level 1, thus the gradient updates across levels mostly reinforce each other instead of conflicting. The Level 2 objective learns category-separable embeddings, while the Level 3 objective further separates root causes within each category. Together, these objectives also help Level 1 detection by creating a more structured embedding space and a cleaner binary decision boundary.

Each level uses a dedicated loss, and we compute all losses at the mini-batch level. Level 1 uses binary cross-entropy, $\mathcal{L} * \text{detect}$, over all samples in the batch. Level 2 uses class-weighted cross-entropy, $\mathcal{L} * \text{cat}$, over fault categories and

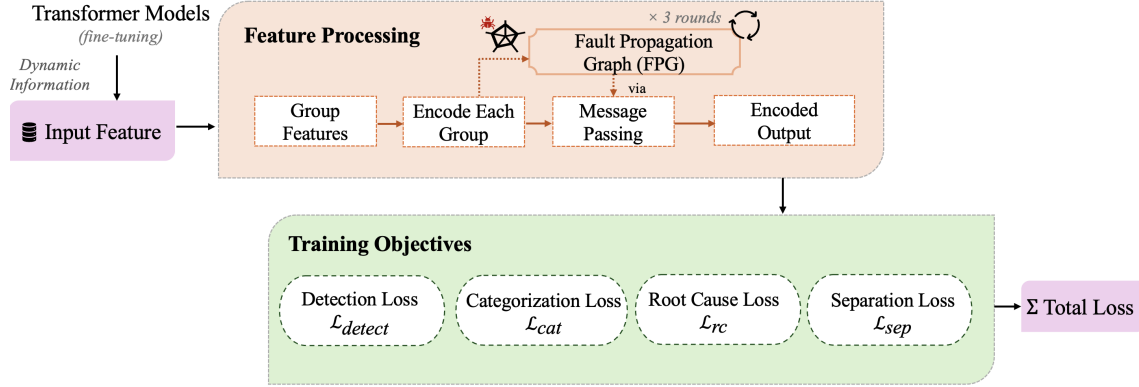


Fig. 10. Training view of DEFault++

applies it only to faulty samples. Level 3 optimizes two objectives: per-category cross-entropy, $\mathcal{L} * rc$, over root causes, and a root-cause separation loss, $\mathcal{L} * sep$, that encourages separation between root causes within the same category.

Cross-entropy trains the per-category heads but does not directly shape the embedding space, so two root causes in the same category can still sit close together in $\mathbf{H}$ and leave the prototype classifier a small margin. We therefore define $\mathcal{L}_{sep}$ with two components. The first is a contrastive term that acts on the flattened group embeddings $\text{vec}(\mathbf{H})$. For each training sample i with category c_i and root cause r_i , we form pairs with the other samples in the same batch that share c_i . A positive pair shares both c_i and r_i , and a negative pair shares c_i but differs in root cause. The term groups positive pairs together and pushes negative pairs apart within a category, preventing separation (e.g., between a QKV root cause and a masking root cause). Following supervised contrastive learning [42], the contrastive term within category c is

$$\mathcal{L}_{ctr}^{(c)} = -\frac{1}{|\mathcal{P}_c|} \sum_{(i,j) \in \mathcal{P}_c} \log \frac{\exp(\text{sim}(\mathbf{h}_i, \mathbf{h}_j)/\tau)}{\sum_{\substack{k: c_k=c_i \\ k \neq i}} \exp(\text{sim}(\mathbf{h}_i, \mathbf{h}_k)/\tau)} \quad (27)$$

where $\mathcal{P}_c$ is the set of positive pairs within category c , $\text{sim}(\cdot, \cdot)$ is cosine similarity on the normalized flattened group embeddings, and $\tau = 0.1$ is the temperature. The denominator excludes the anchor i , following the supervised contrastive convention [42]. It sums over the other samples in the batch that share the anchor's fault category. We use cosine similarity here because the contrastive term compares the direction of two embeddings [32, 92]. The prototype classifier in Equation (26) instead uses squared Euclidean distance, because prototype matching depends on spatial proximity in the grouped embedding space. The joint training objective $\mathcal{L}_{DEFault++}$ sums the three level-specific losses with the separation loss:

$$\mathcal{L}_{DEFault++} = \mathcal{L}_{detect} + \alpha \mathcal{L}_{cat} + \lambda \mathcal{L}_{rc} + \mathcal{L}_{sep} \quad (28)$$

where α and λ are level-specific weighting coefficients. During training, we apply the root-cause separation loss $\mathcal{L}_{sep}$ only to faulty samples within their ground-truth fault category. At inference, Level 3 uses the predicted category from Level 2. The separation loss is defined as,

$$\mathcal{L}_{sep} = \beta \mathcal{L}_{ctr} + \gamma \mathcal{L}_{pm} \quad (29)$$

where $\mathcal{L}_{ctr}$ is the contrastive term from Equation (27), and $\mathcal{L}_{pm}$ is a prototype-matching loss defined as,

$$\mathcal{L}_{\text{pm}} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(-d(\mathbf{H}_i, \boldsymbol{\pi}_{c_i, r_i})/\tau_p)}{\sum_{r' \in \mathcal{R}_{c_i}} \exp(-d(\mathbf{H}_i, \boldsymbol{\pi}_{c_i, r'})/\tau_p)} \quad (30)$$

where $\mathcal{B}$ is the training batch, $d(\cdot, \cdot)$ is the grouped squared distance from Equation (26), $\mathcal{R}_{c_i}$ is the set of root causes within category c_i , and τ_p is a temperature parameter. $\mathcal{L}_{\text{pm}}$ trains $\mathbf{H}$ so that each sample lies closer to its own root-cause prototype than to any other prototype in the same category. The two terms of $\mathcal{L}_{\text{sep}}$ therefore act in the same direction. The contrastive term shapes the embedding space, and the prototype-matching term ties that space to the classifier used at inference, which keeps the group-level distance gap (Equation (31)) aligned with the prediction. We set $\alpha = 1.0$, $\lambda = 1.0$, $\beta = 0.5$, and $\gamma = 0.3$ through grid search, where β weights the contrastive term $\mathcal{L} * \text{ctr}$ and γ weights the prototype-matching term $\mathcal{L} * \text{pm}$ in $\mathcal{L}_{\text{sep}}$.

4.5 FPG-based Explanation

Level 3 predicts the root cause whose prototype is nearest in the grouped embedding space (Section 4.3.3). To explain that prediction, we compare the predicted prototype $\boldsymbol{\pi}_{c, \hat{r}}$ with the nearest alternative prototype $\boldsymbol{\pi}_{c, \tilde{r}}$, where $\tilde{r} = \arg \min_{r \in \mathcal{R}_c \setminus \{\hat{r}\}} d(\mathbf{H}, \boldsymbol{\pi}_{c, r})$. The distance gap between these two prototypes decomposes by group:

$$\Delta_g = d_g(\boldsymbol{\pi}_{c, \hat{r}}) - d_g(\boldsymbol{\pi}_{c, \tilde{r}}), \quad g = 1, \dots, G \quad (31)$$

where $d_g(\boldsymbol{\pi}_{c, r}) = \|\mathbf{h}_g - \boldsymbol{\pi}_{c, r, g}\|^2$. A positive Δ_g means group g is closer to the predicted prototype than to the nearest alternative, which makes it favor the predicted root cause. A negative Δ_g means the group favors the alternative. The total gap $\sum_g \Delta_g$ equals the difference between the two full prototype distances, which makes the decomposition exact. We then normalize the positive contributions into an importance score per group (see Fig. 6):

$$w_g = \frac{\max(\Delta_g, 0)}{\sum_{g'=1}^G \max(\Delta_{g'}, 0)}, \quad g = 1, \dots, G \quad (32)$$

Groups with negative contributions receive $w_g = 0$. The importance scores sum to one and highlight the groups that most support the predicted root cause.

For structural groups (see Table 3), each w_g maps directly to a transformer subsystem. A large w_g therefore identifies the subsystem whose post-message-passing embedding most supports the diagnosis [101]. For the cross-layer observation group (e.g., representation drift), a large w_g suggests that cross-layer drift is part of the diagnostic evidence rather than a localized component effect. For model-wide context groups (e.g., training dynamics, validation performance), a large w_g indicates model-wide deviation that corroborates the component-level diagnosis but does not localize the fault to a specific subsystem. Since w_g is computed from distances in the embedding space produced by message passing, the score for a structural group reflects evidence aggregated along the FPG edges rather than that group's measurements in isolation. Model-wide context groups reflect each group's own embedding without mixing with other groups.

5 Evaluation

5.1 Research Questions

We evaluate DEFault++ on DEFault-bench using nested grouped cross-validation. We evaluate encoder and decoder architectures separately since their feature and label spaces differ.

- **RQ1:** How effective is DEFault++ in detecting, categorizing, and diagnosing transformer faults?

- **RQ₂**: How does DEFault++ compare with existing DL fault-detection techniques?
- **RQ₃**: How do message passing in the Fault Propagation Graph and the root-cause separation loss contribute to diagnostic performance?
- **RQ₄**: How faithful are the FPG-based explanations to the root-cause diagnosis?

5.2 Experiment Setup

We use nested grouped cross-validation with $k = 5$ outer folds. The encoder subset contains 20 model–task pairs (4 models $\times$ 5 GLUE tasks) and the decoder subset contains 12 (3 models $\times$ 4 language modeling tasks). We keep all instances from the same model–task pair in the same outer fold to prevent information leakage from correlated traces. Each encoder fold therefore holds out 4 pairs (20/5), and each decoder fold holds out 2 to 3 pairs (12/5). Per-seed measurements are already averaged into one instance per configuration (Section 3), thus seed is not a separate grouping variable at this stage. The inner split uses `StratifiedGroupKFold` for model selection [97]. The outer loop evaluates generalization on unseen model–task pairs, while the inner loop selects model hyperparameters. We restrict all preprocessing (scaling, thresholding, and standardization) and hyperparameter grid search to the inner training fold. This design trains 25 models per architecture (5 outer folds $\times$ 5 inner folds). We use the same outer-fold assignments for every method so that each is tested on the same held-out samples.² Each outer fold holds out entire model–task pairs rather than random samples. As a result, a fold-level standard deviation would mostly reflect which pairs were held out in that split, rather than variability in model training. We therefore report the model–task-level results directly and include the per-(model, task) mutation-score heatmap (Figure 15, Section Appendix C) to show where DEFault++ generalizes well and where performance is lower.

We evaluate DEFault++ using precision, recall, and F1. Let TP, FP, TN, and FN denote the true positive, false positive, true negative, and false negative counts. We define precision (P) as the fraction of predicted positives that are correct, recall (R) as the fraction of actual positives recovered, and F1 as the harmonic mean of precision and recall. At Level 1, the faulty and correct classes are balanced by construction (Section 3). We therefore report binary F1 directly. At Levels 2 and 3, fault categories and root causes remain imbalanced. We macro-average precision, recall, and F1 across classes [67] to prevent large categories from masking poor performance on small ones. Throughout the rest of the paper, the F1, precision, and recall reported at Levels 2 and 3 are macro-averaged, and we do not repeat the qualifier in tables and prose. We do not report accuracy. On the imbalanced multi-class problems at Levels 2 and 3, a classifier that always predicts the most frequent category can attain high accuracy while failing on minority classes. Accuracy is therefore not a reliable measure of per-class performance [6, 67].

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}, \quad (33)$$

At Levels 2 and 3 we macro-average each of P , R , and F1 across the C classes (11 or 12 fault categories at Level 2, 40 or 45 root causes at Level 3), which gives each class equal weight regardless of support. A true positive for fault detection means that a faulty configuration is correctly identified, regardless of its fault category. A true positive for fault categorization means that the category of the fault is identified correctly, regardless of the root cause. A true positive for root-cause diagnosis requires both the Level 2 category and the Level 3 root cause to be identified correctly in the same inference path.

²Replication package: <https://github.com/SigmaJahan/DEFaultplusplus-Transformer-Debugging>

Baselines. We compare DEFault++ against four prior techniques only on the Level 1 detection task. We evaluate the baselines on the same train/test split as DEFault++. We run the rule-based techniques with their published rule definitions and thresholds after mapping the required observables to the closest available trace features in our dataset. AutoTrainer’s gradient-threshold rules map to our Training dynamics group. DeepDiagnosis adds rules that partially overlap with the Attention and Score groups. For baselines that require training, we train them only on the Level 1 fault-detection task. In each outer fold, we train the baseline on the training split of DEFault-bench and test it on the held-out portion. We use the model types provided in the original replication packages [7, 39, 94, 107]. We keep the original rule thresholds unchanged. Changing them for DEFault-bench would adapt the baselines to our data, rather than evaluate the methods as prior work reported them. We do not compare Level 2 or Level 3 results since their labels do not align with our transformer component categories or root causes (see Table 5).

Table 5. Baseline techniques’ coverage across all three levels

Technique	Approach	Detection	Categorization	Root-Cause
AutoTrainer [107]	Rule-based (5 rules)	✓	N/A	N/A
DeepDiagnosis [94]	Rule-based (8 rules)	✓	N/A	N/A
DeepFD [7]	Flat ensemble (KNN+DT+RF)	✓	N/A	N/A
DEFault [39]	Hierarchical RF	✓	N/A	N/A
DEFault++	Neural hierarchical	✓	✓	✓

- (i) **AutoTrainer** [107] monitors five training symptoms (vanishing gradient, exploding gradient, dying ReLU, oscillating loss, slow convergence) via threshold rules applied during training. We map each rule to the corresponding features in our dataset and apply the published thresholds without recalibration. AutoTrainer was able to produce binary detection decisions but its symptom types do not map to our fault categories.
- (ii) **DeepDiagnosis** [94] extends AutoTrainer to eight symptom rules (adding unchanged weights, saturated activation, and out-of-range output) with a decision tree that maps detected symptoms to seven fix types. We apply the same feature-mapping strategy and thresholds. Its symptom labels likewise do not map to our fault categories.
- (iii) **DeepFD** [7] combines KNN, Decision Tree, and Random Forest classifiers with union voting to diagnose faults from runtime features. Its five diagnosis classes (e.g., incorrect learning rate, wrong activation function) do not correspond to our fault categories. We evaluate DeepFD as a binary classifier for fault detection only.
- (iv) **DEFault** [39] uses a hierarchical Random Forest classifier that combines static and dynamic features to detect and categorize faults across seven coarse DNN-level categories (hyperparameters, activation function, loss function, optimizer, layer configuration, regularization, weights). These categories operate at a different abstraction level. We evaluate DEFault for binary detection only.

5.3 Implementation

We implement the hierarchical model in PyTorch. We use separate MLPs for each feature group with hidden dimension $h = 32$ and a shared projection to $e = 64$ dimensions. We apply three rounds of message passing in the Fault Propagation Graph with the group-level adjacency matrix. Training uses Adam [43] with learning rate 10^{-3} , weight decay 10^{-4} , and batch size 256. We train for up to 150 epochs with early stopping on the validation split. The loss weights are $\alpha = 1.0$, $\lambda = 1.0$, $\beta = 0.5$, and $\gamma = 0.3$, with temperature $\tau = 0.1$. We address fault-category and root-cause imbalance through inverse-frequency weighting in the corresponding classification losses and select a fixed set of random seeds for reproducibility. The replication package [36] contains the implementation, hyperparameter lists, and trained models

(see ?? for the full hyperparameter configuration). We found training loss to converge by $\sim$ epoch 100, at which point validation Detection F1 plateaus while Category F1 continues to improve modestly until early stopping halts training. Training with early stopping under a 150-epoch cap is therefore sufficient for the three-level hierarchy.

5.4 Answering RQ₁: Effectiveness of Diagnosis

To answer RQ₁, we evaluate DEFault++ at the three diagnostic levels: fault detection (Level 1), fault categorization (Level 2), and root-cause diagnosis (Level 3). We evaluate encoder and decoder architectures separately because their feature groups and label spaces differ.

Fault detection (Level 1). Detection recall reached 0.852 on encoders and 0.906 on decoders, which indicates that the model captured most faulty configurations under the evaluated model–task splits (Table 6a). The encoder tasks were the five GLUE classification benchmarks [91] (SST-2, QNLI, RTE, MRPC, and QQP), and the decoder tasks were the four language-modeling corpora (LAMBADA [71], PTB [58], WikiText-2 [61], and OpenWebText [23]) introduced in the benchmark construction (Section 3).

Table 6. Overall DEFault++ performance across three levels

(a) Level 1: Fault Detection			(b) Level 2: Fault Categorization			(c) Level 3: Root-Cause Diagnosis		
Metric	Encoder	Decoder	Metric	Encoder	Decoder	Metric	Encoder	Decoder
F1	0.8259	0.9094	F1	0.8497	0.8679	F1	0.8512	0.8670
Precision	0.8014	0.9125	Precision	0.8662	0.8794	Precision	0.9407	0.9475
Recall	0.8520	0.9064	Recall	0.8403	0.8649	Recall	0.7932	0.8225

Fault categorization (Level 2). DEFault++ achieved F1 of 0.850 on encoders and 0.868 on decoders (Table 6b). Precision and recall remained balanced (0.866/0.840 on encoders, 0.879/0.865 on decoders), which indicates that the model did not trade one for the other. Residual and Positional were the strongest categories (see Table 7a for the per-category breakdown). Residual reached F1 of 0.961 on encoders and 0.995 on decoders, because the residual-stream feature group produced separable skip-connection measurements for the faults in this category. Positional reached 0.919 and 0.927 because positional-sensitivity features distinguished encoding faults from other categories. The weakest categories differed by architecture. In the encoder-only models we evaluated, Variant (0.741) and Kernel (0.771) reported the lowest F1. Among the decoder-only models, FFN (0.718) and KV Cache (0.758) were the most difficult. Variant differed sharply across these two families (F1 0.741 on the encoder-only models vs. 0.948 on the decoder-only models). Both sides have few Variant instances (25 encoder, 23 decoder; Table 2), which rules out sample size as the cause of the cross-architecture gap. In the decoder-only models, Variant likely benefited from cache-related discriminative features that were absent in the encoder-only models.

Root-cause diagnosis (Level 3). DEFault++ achieved F1 of 0.851 on encoders and 0.867 on decoders (Table 6c). Precision exceeded recall on both architectures. This gap indicates that DEFault++ was conservative at the root-cause level. When the model committed to a root cause, the prediction was almost always correct, but recall remained lower for categories with overlapping root-cause feature patterns. The strongest categories for root-cause diagnosis were Positional (0.930 encoder, 0.975 decoder), Masking (0.925 encoder, 0.845 decoder), and Residual (0.908 encoder, 0.958 decoder). Positional root causes produced distinct patterns in positional-sensitivity, attention-entropy, and pre-softmax score features. Residual root causes were distinguished because skip-connection integrity and gradient-norm features produced non-overlapping distributions across root causes within that category. QKV was the weakest category, with F1 of 0.765 on encoders and 0.646 on decoders, because three of its four root causes disturbed the same projection,

Table 7. DEFault++ performance for Level 2 and Level 3 diagnosis (fault-category-wise)

(a) Level 2: Fault Categorization (per-category)

Category	Encoder			Decoder		
	F1	Prec.	Rec.	F1	Prec.	Rec.
Embedding	0.838	0.852	0.829	0.862	0.833	0.897
FFN	0.836	0.807	0.871	0.718	0.673	0.777
Kernel	0.771	0.814	0.740	0.765	0.848	0.700
KV Cache	—	—	—	0.758	0.716	0.822
LayerNorm	0.790	0.775	0.820	0.843	0.937	0.766
Masking	0.917	0.952	0.886	0.890	0.927	0.857
Output	0.889	0.927	0.864	0.975	0.980	0.969
Positional	0.919	0.950	0.891	0.927	0.949	0.911
QKV	0.825	0.863	0.792	0.789	0.772	0.815
Residual	0.961	0.965	0.958	0.995	0.995	0.995
Score	0.861	0.847	0.876	0.946	0.937	0.957
Variant	0.741	0.776	0.717	0.948	0.986	0.914
Avg.	0.850	0.866	0.840	0.868	0.879	0.865

(b) Level 3: Root-Cause Diagnosis (per-category)

Category	Encoder			Decoder		
	F1	Prec.	Rec.	F1	Prec.	Rec.
Embedding	0.879	0.972	0.806	0.877	0.972	0.816
FFN	0.804	0.847	0.778	0.883	1.000	0.803
Kernel	0.814	1.000	0.706	0.747	0.988	0.676
KV Cache	—	—	—	0.881	1.000	0.817
LayerNorm	0.821	0.917	0.749	0.890	0.969	0.839
Masking	0.925	0.986	0.874	0.845	0.902	0.801
Output	0.882	0.958	0.822	0.972	1.000	0.947
Positional	0.930	0.992	0.886	0.975	1.000	0.953
QKV	0.765	0.868	0.694	0.646	0.727	0.592
Residual	0.908	0.927	0.895	0.958	0.961	0.955
Score	0.835	0.879	0.799	0.908	0.926	0.897
Variant	0.799	1.000	0.717	0.824	0.925	0.773
Avg.	0.851	0.941	0.793	0.867	0.948	0.823

alignment, and update-flow measurements simultaneously. Root-cause count alone did not explain the per-category ordering. Residual remained strong with five root causes because its features were well separated, while QKV remained weakest with four root causes because most of them activated the same feature groups. This pattern suggests that per-category difficulty depended more on feature separability than on the number of root causes.

Summary of RQ₁: DEFault++ achieved F1 of 0.826–0.909 for fault detection, 0.850–0.868 for categorization, and 0.851–0.867 for root-cause diagnosis across encoder and decoder architectures. Decoders scored slightly higher than encoders at every level. Level 3 precision exceeded 0.94 on both architectures.

5.5 Answering RQ₂: Comparison with Existing Techniques

To answer RQ₂, we compared DEFault++ with four prior DL fault-detection techniques: AutoTrainer [107], DeepDiagnosis [94], DeepFD [7], and DEFault [39]. We restricted the comparison to Level 1 detection because the baselines do not expose label spaces that match the DEFault++ transformer categories or root causes. Comparing them at Level 2 or Level 3 would equate non-equivalent outputs.

Table 8. Fault-detection comparison between DEFault++ and baselines (F1-score)

Technique	Type	Encoder	Decoder
DEFault++	Neural hierarchical	0.8259	0.9094
DEFault [39]	Hierarchical RF	0.5660	0.7020
DeepFD [7]	Flat ensemble	0.5140	0.6550
DeepDiagnosis [94]	Rule-based (8 rules)	0.3380	0.4820
AutoTrainer [107]	Rule-based (5 rules)	0.2840	0.4270

DEFault++ outperformed all four baselines on both architectures (Table 8). On encoders, DEFault++ reached F1 of 0.826. The strongest baseline at Level 1 was DEFault, which reached F1 of 0.566. DeepFD reached 0.514. The two rule-based methods scored lowest: AutoTrainer achieved F1 0.284 and DeepDiagnosis 0.338. Among decoders, DEFault++ reached F1 0.909, outperforming DEFault (0.702), DeepFD (0.655), AutoTrainer (0.427), and DeepDiagnosis (0.482).

The low baseline scores reflect a mismatch between the failure class our benchmark targets and the failure class each baseline was designed to detect. Each baseline observes model-level or training-level evidence (e.g., loss curves, gradient

statistics, fixed rule symptoms) and does not collect component-level transformer observations, so its effectiveness drops when faults change internal attention behavior without disturbing those measurements. AutoTrainer and DeepDiagnosis check fixed training symptoms such as exploding or vanishing gradients, dying ReLU, and oscillating loss. Many faults in DEfault-bench preserve normal loss curves and finite numerical values while changing internal attention behavior, so these rules do not fire, and the two methods reach F1 of 0.284 and 0.338 on encoders and 0.427 and 0.482 on decoders. DeepFD and DEfault score higher because they observe runtime behavior at the model level rather than against fixed thresholds. Their feature spaces, however, were designed for general DNN faults and do not represent attention distributions, QKV alignment, cache behavior, or component-level propagation, which keeps their F1 below 0.71 on both architectures. The comparison therefore measures how far DNN-level diagnostic features reach on transformer faults rather than how two techniques built for the same problem compare. We do not report Level 2 or Level 3 comparisons. AutoTrainer and DeepDiagnosis generate generic symptom labels that do not map to any fault category in our taxonomy. DeepFD and DEfault use DNN-level fault categories that were defined independently of transformer component boundaries. A numeric comparison at those levels would equate non-equivalent labels. We report baseline coverage in Table 5 instead.

Summary of RQ₂: On Level 1 fault detection, DEfault++ outperformed all four existing techniques. The improvement over existing techniques such as DEfault [39] and DeepFD [7] suggests that many transformer faults in our benchmark could not be reliably identified from overall training behavior alone, and that incorporating component-level information improved detection performance in DEfault++. The lower performance of rule-based techniques such as AutoTrainer [107] and DeepDiagnosis [94] further suggests that fixed training-symptom rules were often insufficient when faults affected internal transformer operations (e.g., projection, cache behavior).

5.6 Answering RQ₃: Ablation

Ablation design. We assess the contribution of each architectural component by systematically removing it from DEfault++ and measuring the performance drop. We ablate two components: message passing in the Fault Propagation Graph and the separation loss $\mathcal{L}_{\text{sep}}$. The separation loss applies only at Level 3. We define two variants for Levels 1 and 2 and four variants for Level 3 as follows.

- (1) **DEfault++**. The complete model with message passing in the FPG and the separation loss at Level 3.
- (2) – **FPG**. We remove message passing in the FPG while retaining $\mathcal{L}_{\text{sep}}$ at Level 3. This variant measures the contribution of propagation-informed feature interaction.
- (3) – **Sep (Level 3 only)**. We remove the separation loss ($\beta = 0, \gamma = 0$) while retaining FPG. This variant measures the contribution of within-category root-cause discrimination.
- (4) – **FPG – Sep**. We remove both FPG and $\mathcal{L}_{\text{sep}}$, retaining only the feature-group MLPs with projection. This variant tests the hierarchical classifier alone.

To test whether the specific FPG topology matters beyond having any graph structure, we define two additional topology variants as follows.

- (1) **Rewired**. We keep the same number of nodes and edges as the FPG but randomly reassign edge endpoints while preserving the degree distribution, following the degree-preserving rewiring procedure of Maslov and Sneppen [59]. This rewiring preserves the overall graph density and local connectivity profile while breaking the meaningful propagation links between transformer components.

- (2) **Random.** We replace the FPG with an Erdős–Rényi random graph [18] with approximately the same edge density. This replacement removes both the meaningful propagation links and the degree structure of the original FPG.

We evaluate the contribution of message passing in the Fault Propagation Graph and the separation loss through ablation. For fault detection & categorization levels, separation loss does not apply, thus we compare DEFault++ *with* & *without* message passing (see Tables 9a and 9b). At Level 3 we compare all four variants (see Table 10). We apply message passing on the Fault Propagation Graph within a shared encoder used across all three tasks (i.e., detection, categorization, and diagnosis), following standard multi-task learning practice with shared representations [8].

• **Fault detection (ablation).** Removing message passing in the FPG modestly reduces detection performance on both architectures (Table 9a). Binary detection is a coarse task. A fault that disrupts any transformer component shifts the overall feature distribution enough for the binary classifier to separate faulty from correct configurations. Message passing adds cross-group information that improves the binary boundary, but the group-level features alone already support detection. We retain message passing at Level 1 because cross-component aggregation helps detect faults with weak local deviations, and the ablation confirms that removing it hurts detection.

Table 9. Ablation studies for Level 1 fault detection and Level 2 fault categorization.

(a) Level 1 fault detection					(b) Level 2 fault categorization				
Arch.	Variant	F1	Prec.	Rec.	Arch.	Variant	F1	Prec.	Rec.
Encoder	DEFault++	0.826	0.801	0.852	Encoder	DEFault++	0.850	0.866	0.840
	–FPG	0.782	0.752	0.816		–FPG	0.742	0.757	0.731
Decoder	DEFault++	0.909	0.912	0.906	Decoder	DEFault++	0.868	0.879	0.865
	–FPG	0.890	0.890	0.891		–FPG	0.788	0.797	0.783

• **Fault categorization (ablation).** We find that removing message passing in the FPG produces the largest drop in the entire ablation (Table 9b). Categorization requires the classifier to distinguish among 11 fault categories on encoders and 12 on decoders, several of which affect overlapping feature groups. A QKV fault propagates to attention scores and the residual stream. A Score fault propagates to attention weights and onward to the residual stream, but through a narrower path than a QKV fault, which simultaneously affects both scores and values. Without message passing, we compute each group embedding independently, and the classifier can only infer cross-group relationships from the flat projection. With message passing, we route feature evidence along the FPG edges before the projection step, producing representations that combine each group’s measurements with those of its structural neighbors. We attribute the large drop to this loss of cross-group structure, because propagation patterns are the distinguishing evidence for categories with overlapping features.

• **Root-cause diagnosis (ablation).** We ablate both the separation loss and message passing independently at Level 3 (Table 10). We find that removing the separation loss produces the larger drop. Root causes within the same fault category share much of the same feature distribution because they affect the same transformer component. A parameter-initialization fault and a dynamic-parameter-registration fault both appear in the QKV feature group with similar aggregate patterns. Cross-entropy alone does not explicitly structure the embedding space to separate these closely positioned root causes. We designed the separation loss to address this gap by pulling same-root-cause embeddings together and pushing different-root-cause embeddings apart within each category.

We find that removing message passing in the FPG at Level 3 (while retaining the separation loss) produces a smaller drop. Message passing aggregates propagation-related evidence across groups before prototype comparison, but the

Table 10. Ablation study of Level 3 root cause diagnosis

Architecture	Variant	F1	Precision	Recall
Encoder	DEFault++	0.851	0.941	0.793
	–FPG	0.812	0.933	0.776
	–Sep	0.756	0.910	0.752
	–FPG–Sep	0.701	0.883	0.690
Decoder	DEFault++	0.867	0.948	0.823
	–FPG	0.833	0.940	0.809
	–Sep	0.781	0.925	0.781
	–FPG–Sep	0.734	0.896	0.721

root-cause discrimination problem centers on within-category separation rather than cross-group structure. We observe that removing both components together produces the largest overall drop, which confirms that message passing and the separation loss contribute complementary information at Level 3.

Topology ablation. The preceding ablation compares the full FPG with no message passing at all. To test whether the specific FPG topology matters, we compare it with two alternative graph structures: Rewired (same density, shuffled edges) and Random (Erdős–Rényi graph with matched density). F1 follows a consistent ordering at every level and on both architectures: DEFault++ > Rewired > Random > –FPG (see Table 11). The FPG with the correct propagation structure outperforms both alternative topologies. Rewired edges preserve the graph density and degree distribution but break the meaningful propagation links, and the performance drops below the full FPG at every level. Random edges lose both the specific links and the degree structure of the original FPG, and performance drops further. Removing message passing entirely (–FPG) produces the lowest performance. The topology effect is strongest at Level 2, where DEFault++ outperforms Rewired by 0.080 (encoder) and 0.061 (decoder) in F1. This is consistent with the earlier finding that fault categorization depends most on cross-group propagation structure. At Level 1 and Level 3, the topology differences are smaller because binary detection is a coarser task and root-cause diagnosis depends more on within-category separation than on cross-group structure.

Table 11. Topology ablation results using F1 across graph variants

Level	Architecture	DEFault++	Rewired	Random	–FPG
L1	Encoder	0.826	0.790	0.786	0.782
	Decoder	0.909	0.894	0.892	0.890
L2	Encoder	0.850	0.770	0.754	0.742
	Decoder	0.868	0.807	0.796	0.788
L3	Encoder	0.851	0.820	0.816	0.812
	Decoder	0.867	0.838	0.835	0.833

We observed that the three diagnostic levels differ in their degree of difficulty. Precision remains consistently high across all levels and for both architectures, whereas recall remains the primary limiting factor at every stage. Recall is lowest at Level 3 root-cause diagnosis (0.793 encoder, 0.823 decoder), especially in categories whose root causes share feature groups; QKV is weakest (F1 0.765 encoder, 0.646 decoder). The RQ₃ ablation points to the same step. Removing the separation loss costs more Level 3 F1 than removing message passing does, which indicates that within-category root-cause separation is the step that depends most on the technique’s targeted mechanism. Detection and categorization appear to be comparatively well supported in our setting. One possible explanation is that faults affecting monitored components alter the overall feature distribution, which makes detection easier. For categorization,

the results suggest that FPG message passing helps recover cross-component propagation patterns that distinguish categories with overlapping symptoms.

Summary of RQ₃: Level 3 within-category root-cause separation remains the most challenging part of the diagnosis task, and the separation loss is the component that most improves this capability. In contrast, FPG message passing contributes most at Level 2, where categories exhibit overlapping symptoms and require propagation-aware reasoning across components. The topology ablation further showed that the diagnostic performance depends on the specific FPG structure rather than graph connectivity alone, supporting the usefulness of the FPG representation for categorization. However, the ablation does not establish whether the learned edges reflect the actual fault-propagation paths of the trained model.

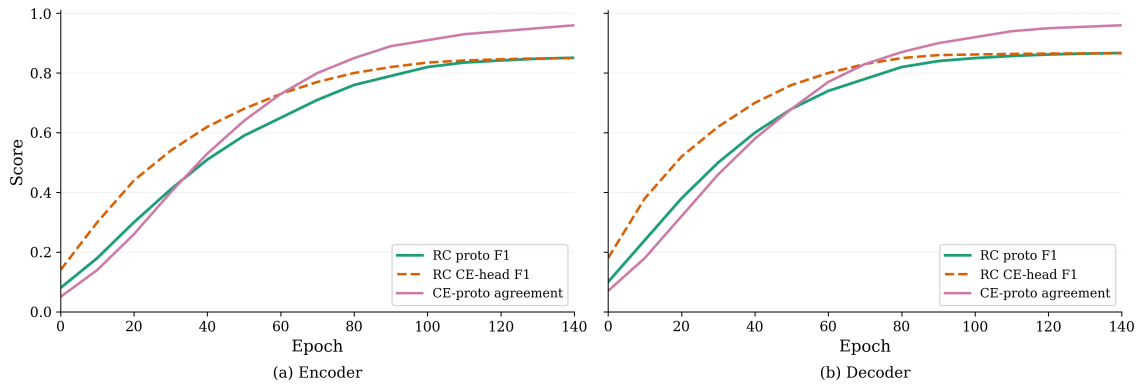


Fig. 11. Prototype-label agreement during training

5.7 Answering RQ₄: Explanation Faithfulness

To answer RQ₄, we evaluated whether the feature groups identified by DEFault++ supported the predicted root cause. We used the prototype-based explanation from Level 3. For each instance, DEFault++ compares the predicted root-cause prototype with the nearest alternative prototype and decomposes the distance gap by feature group (see Equation (31)). DEFault++ trains with two complementary objectives at Level 3. The cross-entropy heads classify root causes from logits, while the prototype-matching loss (see Equation (30)) brings each sample closer to the correct prototype and further away from alternatives within the same fault category. We use the prototype classifier for explanation because the distance to a prototype decomposes group by group (see Equation (31)), which gives each feature group its own contribution score. Cross-entropy logits produce a single prediction vector and do not offer this per-group breakdown. CE-prototype agreement, defined as the fraction of samples for which the cross-entropy head and the prototype classifier predicted the same root cause, increased steadily over training alongside the cross-entropy objective and the root-cause prototype F1 (see Figure 11 for the training dynamics). The cross-entropy F1 increased faster in early epochs because logit-based supervision provided a direct gradient, while the prototype-matching loss required the embedding clusters to separate first. The two curves converged by mid-training, which indicates that the cross-entropy and prototype-matching objectives produced consistent predictions despite optimizing different loss surfaces. At inference time, we only used the prototype-based classifier.

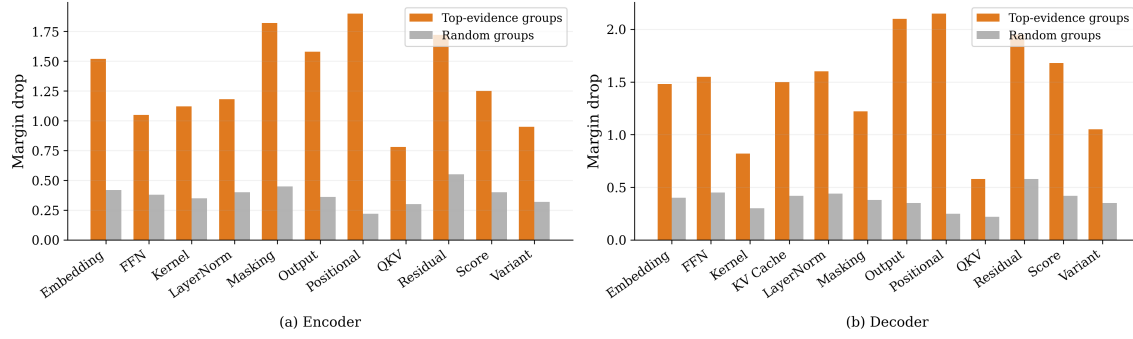


Fig. 12. Group ablation analysis for the FPG-based explanation

We also tested the explanation through group ablation. For each sample, we identified the two groups with the largest importance scores (Equation (32)). The importance scores are normalized to sum to one, and the distribution is typically concentrated on a few groups. In our experiment, the top two groups captured a substantial share of the total score. We zeroed out these two groups and compared the drop in the prototype margin (the gap between the predicted and the nearest alternative prototype) with the drop obtained by zeroing two randomly chosen groups. Removing the most important groups reduced the gap more than removing random groups across all evaluated categories on both architectures (Figure 12). This pattern is consistent with the highlighted groups carrying the evidence that drives the prototype decision. Among categories, QKV shows the smallest margin drop on both architectures, which is consistent with QKV being the hardest category to separate (F1 0.765 on encoders and 0.646 on decoders). The prototype decision there starts from a smaller margin, so it has less to lose under ablation. The ablation evaluates internal faithfulness by measuring agreement between the explanation and the prototype decision. Since feature-group explanations in DEFault-bench do not have directly observable ground truth, the ablation study focuses on consistency with the model’s decision process (i.e., faithfulness of the explanation) and does not assess alignment with the underlying causal mechanism of the original fault.

Summary of RQ₄: The FPG-based explanations are internally faithful to the prototype-based root-cause decision under the evaluated ablation. Removing the most important feature groups reduced the prototype margin more than removing random groups. The evidence covers internal faithfulness, and external causal attribution would require independent feature-group ground truth that the benchmark does not provide.

6 Real-world Fault Evaluation

We use real-world GitHub issues to evaluate whether DEFault++ generalizes beyond the synthetic faults from our mutation-based benchmark. Following prior work [38, 64, 96], we collected 20 transformer-related bug-fix issues from open-source repositories and successfully reproduced 11 of them. For each reproduced issue, we compared the faulty version with the fixed version using the same feature extraction technique used in the experimental evaluation. DEFault++ detected 8 of the 11 faults, identified the correct fault category for 8, and identified the root cause for 4 (see Table 12). Fault detection (FD) checks whether DEFault++ marks the faulty version as faulty. Fault categorization (FC) checks whether it assigns the fault to the correct transformer category. Root-cause diagnosis (RC) checks whether the predicted cause matches the cause confirmed by the bug fix of the source issue.

Table 12. Real-world evaluation of DEFault++ on 11 transformer faults

Source	Fault category	DEFault++ result			Feature Importance (top)	Relevant Scope of DEFault++
		FD	FC	RC		
jax#23349	Masking	✓	✓	✓	Attention: padding attention mass	Attention feature group
pytorch#103082	Masking	✓	✓	✓	Attention: future attention mass	Attention feature group
transformers#19045	Positional	✓	✓	✗	Positional: positional sensitivity	Forward sequential propagation (M1)
transformers#17886	Positional	✓	✓	✗	Attention: inter-head cosine similarity	Forward sequential propagation (M1)
ai-edge-torch#6	QKV	✓	✓	✓	Score: pre-softmax score	Simultaneous propagation (M2)
nsa-pytorch#20	KV Cache	✓	✓	✗	KV Cache: cache-related behavior	Generation-time cache update path
transformers#37574	KV Cache	✗	✗	✗	–	Generation-time cache behavior
transformers#36096	Score	✗	✗	✗	–	Runtime failure before feature extraction
pytorch#116333	Kernel	✗	✗	✗	–	Backend stride validation
transformers#35896	Variant	✓	✓	✗	Attention: attention entropy	Cross-layer propagation (M4)
diffusers#11903	QKV	✓	✓	✓	Optimization: projection update ratio (Dynamic parameter registration)	QKV projection/update path

Summary: FD - 8/11, FC - 8/11 categories, RC - 4/11 root causes.

FD = fault detection, FC = fault categorization, RC = root-cause diagnosis. **Crash*** indicates an explicit runtime failure before feature extraction. The summary counts all 11 cases. transformers#36096 crashed before feature extraction and is treated as not applicable to DEFault++ rather than a diagnostic failure.

We observed two masking faults where the faulty versions assigned attention to tokens that should have been masked. DEFault++ captured these faults through padding attention mass (jax#23349) and future attention mass (pytorch#103082), and both values returned to zero after the fixes. We also observed the QKV fault in diffusers#11903 through a near-zero QKV update ratio. In that issue, LoRA updated stale projection modules while the forward pass used the fused QKV projection, and the updated parameters did not affect the model output.

We also found four faults where DEFault++ detected the fault through feature groups connected in the Fault Propagation Graph (FPG). In transformers#19045 and transformers#17886, the Positional faults changed how the model used token positions, which then affected attention behavior. DEFault++ therefore assigned higher importance to positional sensitivity and head similarity. In ai-edge-torch#6, the checkpoint loader assigned QKV projection weights incorrectly, which changed the pre-softmax score distribution. DEFault++ captured this effect through the Score feature group, which the FPG connects to QKV projection by simultaneous propagation (M2).

In transformers#35896, sliding-window attention was applied to the wrong layers. Sliding-window and full attention produce different per-layer entropy profiles, because the sliding window restricts each query to a local context and lowers attention entropy at the affected layers. DEFault++ flagged the fault through this per-layer entropy deviation. The cross-layer propagation mechanism (M4) is consistent with the case, because the misconfigured layers' representations move downstream through the residual stream. The localizing evidence, however, is the per-layer entropy contrast itself rather than cross-layer propagation. Across these four cases, DEFault++ identified the faulty component through a feature group connected to it in the FPG. These cases indicate that the FPG-based explanation can help interpret real-world faults when the fault affects connected feature groups.

In nsa-pytorch#20, DEFault++ correctly predicted the faulty status and the fault category (i.e., KV Cache) but incorrectly predicted the root cause as cache invalidation. However, the ground truth is update synchronization, suggesting current token's key-value vectors were not synchronized with the attention and prediction computation.

DEFault++ could not diagnose three real-world faults across all three levels. In transformers#37574, DEFault++ missed the fault as the faulty behavior appeared during autoregressive generation, where cached key-value states from earlier decoding steps interact with the current token. DEFault++ captures cache-related behavior during fine-tuning, but it does not trace each cached decoding step during generation, which may miss abnormal behavior that appears only at generation time. In pytorch#116333, the fault came from backend stride validation rather than transformer-layer

behavior. DEFault++ traces model-level fine-tuning dynamics. It does not capture low-level kernel-dispatch constraints. Detecting this fault would require system-level information beyond the model trace used by DEFault++. Finally, in transformers#36096, the faulty `flex_attention` implementation returned an attention tensor with an invalid rank. DEFault++ could not extract diagnostic features since the run failed before producing a valid model trace. We therefore treat this case as not applicable to DEFault++, rather than as a diagnostic failure.

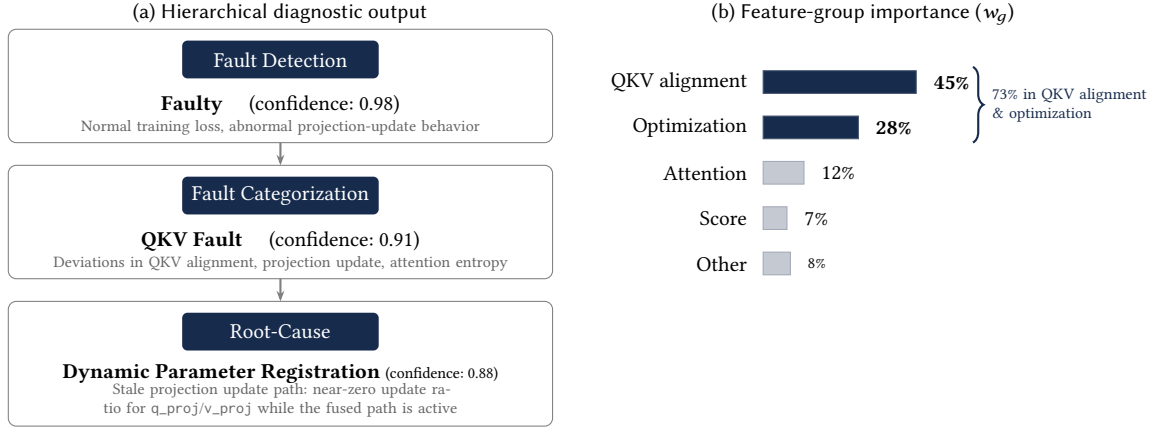


Fig. 13. DEFault++ diagnosis and feature-group importance for the stale QKV fusion fault in Listing 2.1

The dynamic parameter registration root cause covers a family of faults in which parameter updates and forward-path participation are misaligned. The DEForm operator QFG injects one sub-mechanism, freezing gradients while the parameters remain in the forward path. The `diffusers#11903` fault is the opposite sub-mechanism, where gradients flow but the parameters are removed from the forward path. DEFault++ was trained on the first sub-mechanism but identifies the second one through the same QKV update-activity signature. This generalization suggests that the root-cause label spans sub-mechanisms within a category rather than overfitting to a single operator.

We use `diffusers#11903` as a representative example of a silent QKV fault, which was also used in the motivating example (see Section 2). In this case, training completed normally, but LoRA updated projection parameters that were no longer used by the forward pass. The faulty implementation fused the query, key, and value projections into `to_qkv`, while the original projection modules remained accessible to LoRA (see Figure 13).

Level 1: Fault detection. DEFault++ classified the case as faulty even though the training loss curve decreased normally, and the strongest signal came from the optimization feature group where the update ratios for the stale projection modules were near zero relative to the active fused path.

Level 2: Fault categorization. DEFault++ assigned the fault to the QKV category, with the QKV alignment feature group contributing the most and the attention score feature group providing secondary evidence.

Level 3: Root-cause diagnosis. DEFault++ identified the root cause as dynamic parameter registration. The trained QFG operator captures the inverse sub-mechanism (frozen gradients, with parameters still in the forward path), while the `diffusers#11903` fault is the opposite sub-mechanism (active gradients, with parameters removed from the forward path). DEFault++ identifies the shared dynamic parameter registration signature through the QKV alignment and projection update-activity features. The QKV alignment and Optimization feature groups together explain 73% of the normalized feature importance (Figure 13). This aligns with the repair decision because the fault affects both the

projection path and the parameter updates. The repair ensures that LoRA updates the projection module used in the forward pass rather than the stale modules left after QKV fusion.

7 Developer Study

We evaluated whether DEFault++ diagnoses help developers choose correct repair actions for transformer faults. Participants completed four debugging scenarios derived from reproduced real-world GitHub issues. For each scenario, they selected a repair action either from baseline materials alone or from the same materials plus the DEFault++ diagnosis. We measured repair-action accuracy, confidence, self-reported time, and perceived usefulness.

Study Design. We used a within-subjects, counterbalanced design with two conditions [100]. In the *baseline* condition, participants received a scenario description, observed behavior, and logs with metrics. In the *DEFault++ assisted* condition, participants received the same materials plus the DEFault++ diagnosis: Level 1 fault status, Level 2 fault category, Level 3 localized root cause, and supporting feature-group evidence. Each participant completed all four scenarios, two per condition. We counterbalanced condition assignment across two survey forms. This scenario-based evaluation follows prior developer studies of DL debugging techniques [57, 83]. We derived the four scenarios from reproduced real-world GitHub issues (see Section 6), each representing a distinct transformer fault category (Table 13). For three of the four scenarios (S2, S3, and S4), DEFault++ produced a correct Level 1 detection and Level 2 categorization but an incorrect Level 3 root cause (see Table 14 for the Level 3 prediction shown to participants in each scenario). Only S1 received a fully correct three-level diagnosis. We retained these scenarios to test whether category-level diagnostic support helps developers choose repair actions even when the root-cause prediction is imperfect. Each scenario asked participants to choose one repair action from four options: the correct repair from the source issue, two incorrect repairs targeting related components, and one unrelated hyperparameter change. The task measured whether participants could translate the available diagnostic evidence into the correct repair direction. The primary outcome was repair-action

Table 13. Developer-study scenarios and correct repair actions

Scenario	Source Issue	Category	Repair Action
S1: Cached decoding visibility	pytorch #103082	Masking	Fix the causal mask for cached decoding so that the new token attends to the correct cached key positions.
S2: Position-dependent scoring drift	transformers #19045	Positional	Recompute relative-position distances from the true token positions during cached decoding.
S3: Incremental state update mismatch	nsa-pytorch #20	KV Cache	Write the current token's key-value vectors into the cache before computing attention and prediction scores.
S4: Local-attention layer assignment	transformers #35896	Variant	Fix the layer-selection condition so that local/sliding-window attention is applied to the intended layers.

Table 14. DEFault++ diagnosis shown to participants in each developer-study scenario

Scenario	Level 1	Level 2	Level 3 (predicted)	Level 3 (ground truth)
S1 (Masking)	Faulty ✓	Masking ✓	Dynamic mask ✓	Dynamic mask
S2 (Positional)	Faulty ✓	Positional ✓	Indexing ✗	Relative position
S3 (KV Cache)	Faulty ✓	KV Cache ✓	Cache invalidation ✗	Update synchronization
S4 (Variant)	Faulty ✓	Variant ✓	Dynamic dispatch ✗	Variant configuration

correctness, scored against the ground-truth fix from the source issue. We also collected self-reported confidence on a 5-point scale and self-reported time as a perceived-effort measure. After completing all scenarios, participants rated the clarity, usefulness, and practical value of the DEFault++ diagnosis on 5-point Likert scales and answered two preference questions comparing the baseline and assisted conditions.

Participants. The Dalhousie University Research Ethics Board³ reviewed and approved the developer study. Eligibility required experience with transformer architectures in at least one of training, fine-tuning, inference, debugging, or evaluation [57, 83]. We recruited 21 practitioners through graduate student mailing lists, academic networks, and direct outreach. Participation was anonymous, voluntary, uncompensated, and hosted through Microsoft Forms.

Table 15. Developer study: participant demographics and repair outcomes (N = 21 participants)

(a) Participant demographics			(b) Repair accuracy and confidence by scenario and condition					
Characteristic	Value	N	Scenario	Condition	N	Correct	Accuracy	Confidence
ML/DL experience	<1 year	1	S1 (Masking)	Baseline	12	8/12	66.7%	3.75
	1–2 years	6	S1 (Masking)	Assisted	9	9/9	100.0%	4.11
	3–5 years	10	S2 (Positional)	Baseline	9	4/9	44.4%	2.78
	>5 years	4	S2 (Positional)	Assisted	12	10/12	83.3%	3.75
Prior transformer debugging Experience	Yes	15	S3 (KV Cache)	Baseline	12	7/12	58.3%	3.50
	No	6	S3 (KV Cache)	Assisted	9	8/9	88.9%	4.11
Current Role (multi-select)	Research assistant	10	S4 (Variant)	Baseline	9	5/9	55.6%	3.33
	Graduate student	11	S4 (Variant)	Assisted	12	8/12	66.7%	4.08
	Industry practitioner	9						
			Total	Baseline	42	24/42	57.1%	3.38
			Total	Assisted	42	35/42	83.3%	4.00

Results. We report descriptive statistics for each scenario and condition, following prior vignette-based debugging studies such as UMLAUT [83] and KUnit [57]. As each repair-action response is selected from four options and each participant sees only two scenarios per condition, we treat the per-condition difference as an effect-size estimate rather than as the basis for a hypothesis test.

Repair accuracy. Across the four scenarios, participants selected the correct repair action in 24 of 42 baseline responses (57.1%) and 35 of 42 DEFault++ assisted responses (83.3%), which is an increase of 26.2% (see Table 15). The largest improvement occurred for the positional fault, where baseline accuracy was lowest. The smallest improvement occurred for the variant fault, where the scenario already provided a clearer symptom-to-repair path. Under the assisted condition with DEFault++, no participant selected the unrelated hyperparameter option (see Figure 14). Three of the four assisted-condition scenarios (S2, S3, and S4) presented an incorrect Level 3 root cause to participants (Table 14). Assisted repair accuracy on these three scenarios was 83.3%, 88.9%, and 66.7%. This pattern suggests that the correct Level 2 category and the feature-group evidence together were enough for most participants to choose the correct repair direction.

Confidence. Participants also reported higher confidence under the assisted condition ($M = 4.00$, $SD = 0.83$) than under the baseline condition ($M = 3.38$, $SD = 1.08$). Confidence remained higher for correct responses than incorrect responses in both conditions. We collected self-reported time as a perceived-effort measure, but we do not report it as a task-duration result because the vignette design does not reflect realistic debugging duration (Section 8).

Participant feedback. Participants rated the diagnostic output positively. Twenty participants (95%) agreed or strongly agreed that the output from DEFault++ was easy to understand ($M = 4.14$, $SD = 0.48$), and 19 (90%) agreed or strongly agreed that it helped them decide what repair to try first ($M = 4.29$, $SD = 0.96$). The same proportion reported that they would use a tool like DEFault++ in practice ($M = 4.10$, $SD = 0.89$). 15 participants (71%) preferred the assisted condition for debugging decision support.

³<https://www.dal.ca/research-and-innovation/support-for-researchers/responsible-conduct-research/human-ethics.html>

Within-subjects comparison. At the participant level, seven participants showed improved performance under the assisted condition, 14 showed no change, and none declined.

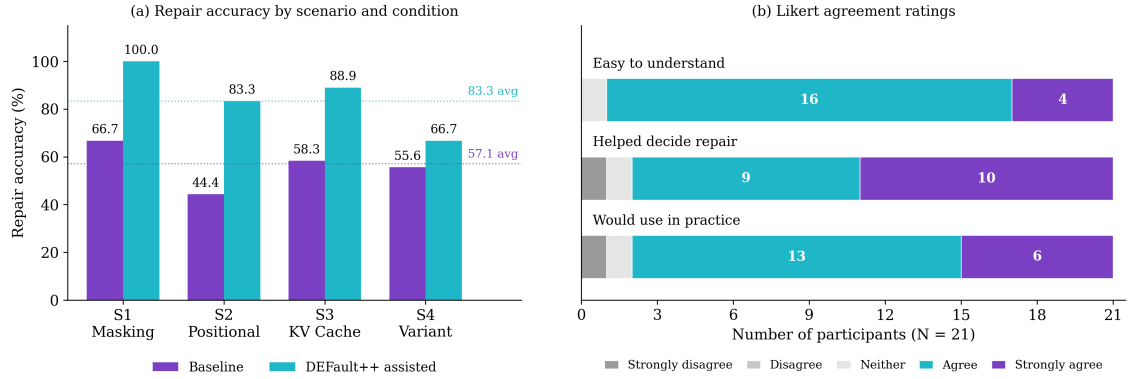


Fig. 14. Developer study results. (a) Repair accuracy by scenario and condition. (b) Likert agreement ratings.

Discussion. DEFault++ helped participants most when the observed symptoms did not clearly indicate the correct repair. In S2, S3, and S4, DEFault++ predicted the correct Level 2 category but the wrong Level 3 root cause. Even under these conditions, participants in the assisted setting repaired faults more accurately than participants in the baseline setting. These results indicate that participants relied on the Level 2 category and the accompanying feature-group evidence to narrow the repair space. Incorrect Level 3 predictions reduced diagnostic precision but the assistance remained useful. We did not systematically vary diagnostic-error types in this study. We therefore cannot yet determine how different Level 3 errors affect repair performance.

Participants in the baseline setting rarely selected unrelated repairs. Instead, they usually chose repairs that targeted plausible but incorrect transformer components. This behavior indicates that participants could reason about the debugging context but could not reliably distinguish between related transformer components using symptoms alone. DEFault++ exposed evidence tied to specific fault categories and parameter groups. Participants also reported higher confidence in the assisted setting across scenarios. These findings align with prior developer studies that showed similar effects from diagnostic support. UMLAUT helped developers identify and repair more faults [83]. Participants in the KUnit study reported that mocks improved independent component testing [57].

8 Threats to Validity

Threats to internal validity concern experimental errors [100]. Information leakage across configurations that share the same model–task pair could threaten internal validity, since it would let the diagnostic model match a held-out trace to correlated training traces rather than learn fault-induced differences. We mitigate this threat through nested grouped cross-validation, which keeps all instances from one model–task pair in the same outer fold, fits preprocessing and selects hyperparameters on the inner training data alone, and reuses the same outer-fold assignments across all compared methods. The joint training objective could pose a second internal threat. Because the shared encoder couples the losses, optimization pressure from one level might change the representations used by the others. The separation loss is the main concern, since it targets root-cause discrimination and could steer the shared representation in ways that do not benefit Level 1 or Level 2. To mitigate this threat, we apply the separation loss only to faulty samples and

only within their ground-truth fault category (Section 4.4), which confines its influence to within-category structure rather than the global representation. We quantify the cross-level effect through the RQ₃ ablation.

Threats to construct validity concern whether the benchmark and the diagnostic outputs reflect the intended diagnosis problem [84]. Generating DEFault-bench through mutation operators rather than naturally occurring faults could threaten construct validity [31, 37], since the injected faults might not fully reflect the faults developers encounter in practice. We mitigate this threat by deriving each operator from a documented transformer root cause. The attention-internal categories are based on the 555-fault attention taxonomy [37], and the non-attention categories are based on prior DNN fault taxonomies mined from open-source projects [30, 33].

Class imbalance across categories and root causes could pose a second construct threat, since frequent classes might dominate the aggregate scores. We address this threat through macro-averaged F1 and inverse-frequency class weighting [6]. Taxonomy drift could pose a related threat, as attention variants, caching schemes, and kernel implementations evolve and new fault types might fall outside the current taxonomy. We mitigate this risk by mapping each operator to the attention taxonomy [37] and the established DNN fault taxonomies [30, 33], hence a new operator can be added as these taxonomies grow without changing the diagnostic technique. DEForm includes 52 operators that cover the root causes reported at the time of benchmark construction [31, 33].

The mutation-killing decision raises a further construct threat specific to DEForm. Our kill criterion uses a one-sided sign-flip permutation test with five matched seeds at $\alpha = 0.05$, whereas DeepCrime uses a generalized linear model with $n = 20$ [31]. With five seeds, a mutant is killed only when all five move the metric in the same direction; thus, a fault with a weaker or noisier effect could survive even when the injected change is present. Our mutation scores are therefore conservative, and a larger seed budget would likely raise them. We accept the tradeoff since it protects label integrity. Every *faulty* instance reflects a statistically distinguishable behavior change, and every *correct* instance reflects a clean run whose perturbation passes the same test (Section 3.5). We chose five seeds as the smallest design that supports an exact test at $\alpha = 0.05$ within our GPU-hour budget.

Trivial mutants could pose a related construct threat, since their effects are large enough that most runs reveal them and might inflate the diagnostic scores without testing fine-grained discrimination [31]. We reduce this risk by sampling severity and layer indices uniformly across {low, medium, high}. We report per-category mutation scores in Figure 5.

A final construct threat concerns the causal interpretation of the diagnostic outputs. First, the training labels in DEFault-bench are causally established by construction. Each mutant differs from its paired clean counterpart through a targeted modification under identical seeds and training conditions, so differences between the paired runs are attributable to the injected change and the killed-mutant label reflects that change. Second, neither the Fault Propagation Graph (FPG) nor the diagnostic model provides causal validation. The FPG is derived from the transformer computation graph, and the diagnostic model predicts labels generated by mutation, so both capture statistical associations rather than causal mechanisms. The RQ₄ group ablation and the trace-to-code checks in the real-world evaluation (Section 6) show consistency with the predicted feature groups but do not establish causal mechanisms. We therefore treat FPG edges as plausible propagation paths in line with Section 4.1.

Threats to external validity concern generalization beyond the studied models and tasks [21]. Model-task overfitting could threaten external validity, since the reported scores might then reflect memorized model-task pairs rather than transferable fault patterns. The nested grouped cross-validation that controls leakage above also mitigates this threat, since holding out entire model-task pairs keeps each evaluation fold to pairs unseen during training and hyperparameter selection. As a result, fold-to-fold variation mainly reflects which pairs were held out rather than training noise, so we

report model–task-level results and visualize generalization behavior in the per-(model, task) mutation-score heatmap (Figure 15, Section Appendix C). Encoder pairs cluster at high mutation scores, with DistilBERT reaching 100% on four of five GLUE tasks and the lowest encoder pair, BERT–SST-2, at 51%. Decoder pairs show larger variance, and LAMBADA yields the lowest scores across all three decoders (25–32%), consistent with its higher seed-to-seed perplexity variance on long-context word prediction [71].

Baseline incompatibility with our label space could pose a second external threat. Because existing baselines do not always produce outputs aligned with our fault categories, a multi-level comparison might conflate non-equivalent outputs. We therefore restrict direct comparison to Level 1 detection and report baseline coverage separately in Table 5. Between-participant variation in the developer study (Section 7) could pose a related external threat. We mitigate this threat through a within-subjects, counterbalanced design in which each participant completes tasks under both the baseline and the DEFault++-assisted conditions, although participants do not solve the same scenario in both conditions.

9 Limitations and Future Work

Single-fault scope. Following prior deep learning mutation studies [28, 31], we inject one fault per training run, which lets us attribute the observed behavior to a single injected fault, necessary for fault categorization and root cause analysis. As a result, DEFault++ leaves out programs that contain several co-occurring faults. Future work can scale the benchmark to multi-fault configurations and extend DEFault++ to compound-fault diagnosis, where it should also reason about how co-occurring faults interact.

FPG is a structural prior. We build the Fault Propagation Graph (FPG) from the transformer’s forward and backward computation. Each edge shows which components a perturbation can reach, but not how strong that dependency is or what kind it is. The model learns a predictive mapping over this fixed structure, and it does not measure how much a fault at one component changes another. Future work can calibrate the FPG against fault-injection measurements and label each edge with a dependency type. The model could then learn how strongly one component affects another and through which mechanism, rather than treating every edge as an identical, unweighted connection.

Architecture coverage. We evaluate four encoder-only and three decoder-only models (66M–125M parameters, 6–12 layers) (Equation (3), Figure 2). DEFault-bench is architecture-family dependent. Across the encoder-only and decoder-only families it covers, the benchmark includes multiple models, tasks, and fault variants, so its results are not tied to any single model. We do not cover encoder-decoder, vision, multimodal, or mixture-of-experts architectures, or models above 125M parameters, thus generalization to them remains unverified. Since each operator targets a component type rather than a specific model, a new encoder-only or decoder-only model can be added to DEFault-bench by mapping its modules to those component types. The trained classifier weights, however, are specific to the seven evaluated models and would need retraining, and we have not tested this transfer on a held-out architecture. Future work can extend both the benchmark and the diagnostic technique to the remaining families.

Developer study. The sample size of our developer study is modest ($N = 21$), and participants worked through vignettes rather than a live debugging environment. Thus we treat self-reported time as a perceived-effort measure rather than a task duration. A larger live debugging study that varies diagnostic error type and severity can better clarify how practitioners may use DEFault++.

10 Related Work

Fault Taxonomies. Empirical studies of Deep Learning (DL) programs have classified faults by root cause [30, 33], covering training, hyperparameter, and layer-configuration faults across feed-forward, convolutional, and recurrent architectures. These taxonomies were proposed before the wide adoption of attention-based architectures and do not include transformer-specific fault patterns. The attention fault taxonomy [37, 38] addresses this gap by classifying 555 attention-related faults into seven categories and 25 root-cause mechanisms. DEFault++ combines these attention categories with five non-attention categories drawn from the prior DNN taxonomies above (see Section 3.1).

Table 16. Comparison of the DEFault-bench mutation process with existing mutation techniques

Dimension	DeepMut++	DeepCrime	DeepFD	PyTorchFI	DEFault-bench
Fault granularity	Layer/param	Source (AST)	Program	Bit/tensor	Transformer unit
Mechanism count	17 ^a	24 impl ^b	5 types	Bit-flip	12 categories
Architecture coverage	FFNN/CNN/RNN	Generic DNN	Generic DNN	CNN (infer.)	Encoder + decoder
Attention mechanisms	✗	✗	✗	✗	✓
Behavior change	✗	✗	✗	✗	✓
Decision basis	Accuracy	GLM + d	Accuracy	SDC rate	Accuracy + sign-flip
Noise control	✗	✓ ^c	✗	✗	✓
Scale (reported)	Varies	1,760	52 programs	10 ⁷ + inj.	5,556
Labeled traces	✗	✗	✓	✗	✓

^a8 source-level + 9 model-level mechanisms [28]. ^b35 defined, 24 implemented from real faults [31]. ^cDeepCrime uses GLM with $n=20$ retrainings and Cohen's d for statistical killing. ✓ = supported; ✗ = not supported.

Mutation Testing for Deep Learning. Mutation testing is a well-established technique in software engineering for assessing test quality, typically by introducing small, controlled changes to a program and checking whether existing tests detect them (see Table 16). Several researchers have adapted this idea for deep learning. DeepMutation [54] and DeepMutation++ [28] introduced mutation operators that modify training code, model structure, or learned parameters, and use mutation scores to quantify test adequacy for neural models. A major challenge in this setting is training stochasticity. The same injected change can appear harmful under one random seed and negligible under another, and seed-induced variance can even rival the effect size of subtle faults [65]. Probabilistic mutation testing addresses this by treating mutation killing as a statistical decision over repeated runs [87]. DeepCrime [31] adopts a similar strategy, performing multiple retrainings and applying statistical tests with effect-size thresholds to determine whether a mutation has a significant impact.

A smaller number of studies use injected faults explicitly for fault diagnosis. DeepFD [7] injects common fault types into deep learning programs and trains classifiers on runtime features to identify fault categories. Subsequent work extends this idea to larger corpora of faulty programs through data-driven localization [95]. These approaches remain limited for transformer architectures for two reasons. First, they target generic model and training faults (e.g., loss misconfiguration, incorrect layer numbers) rather than faults within the transformer architecture. Second, these works rely on high-level model performance (e.g., accuracy drop) and standard runtime features that do not capture the internal structure of attention-based models.

Debugging Techniques for Deep Learning Programs. Recent work has framed DL fault diagnosis as a classification problem using runtime information collected during training. DeepFD [7] extracts features from loss curves, gradients, weight statistics, and activation patterns, and trains a classifier to identify common training faults such as an incorrect loss function, learning rate, or activation function. DEFault [39] extends this idea into a hierarchical classifier that first detects whether a program is faulty and then assigns the fault to one of seven training or model fault categories. DEFault++ inherits this hierarchical structure, but changes the diagnostic target and the evidence source. It replaces

architecture-agnostic DNN features with measurements taken from transformer components and adds root-cause diagnosis within each predicted category. As a result, DEFault++ can distinguish faults in QKV projection, masking, attention scoring, residual connections, and other transformer-specific components, whereas DeepFD and DEFault report generic DNN fault categories. Among prior fault-diagnosis techniques, only DEFault++ provides fault detection, categorization, root-cause diagnosis, attention-awareness, and a built-in explanation together (Table 17).

Table 17. Comparison of fault diagnosis techniques for DL/DNN programs

Technique	Target	FD	FC	RC	Attn-Aware	Explanation
AutoTrainer [107]	DNN	✓	Partial (5)	✗	✗	Rule-based
UMLAUT [83]	DNN	✓	Partial	✗	✗	Rule-based
DeepDiagnosis [94]	DNN	✓	✓ (8)	✗	✗	Decision tree
DeepLocalize [96]	DNN	NaN/Inf only	✗	✗	✗	Numerical trace
DeepFD [7]	DNN	✓	✓ (5)	✗	✗	Feature attribution
DEFault [39]	DNN	✓	✓ (7)	✗	✗	SHAP
ATTNChecker [49]	Transformer	NaN/Inf only	✗	✗	✓	✗
AtPatch [99]	Transformer	Map outlier	✗	✗	✓	✗
FT-Transformer [12]	Transformer	Soft errors	✗	✗	✓	✗
DEFault++ (Ours)	Transformer	✓	✓ (12)	✓ (45) [†]	✓	FPG+prototype

FD = fault detection; FC = fault categorization; RC = root-cause diagnosis; [†]DEFault++ covers 45 root causes in total (40 apply to encoders, all 45 apply to decoders)

A second category of work uses predefined rules or localization heuristics. The closest neighbors to DEFault++ in this family are AutoTrainer [107], UMLAUT [83], and DeepDiagnosis [94], all of which monitor training-time symptoms such as exploding gradients, vanishing gradients, and oscillating loss. Numerical-instability detectors such as DeepLocalize [96] and GRIST [104] look for NaN or Inf values, while shape and invariant checkers such as Tensfa [102] and TFCheck [4] validate tensor shapes and training-process invariants. A second cluster operates at finer granularity: MODE [55] and DeepFault [17] localize faults at the neuron level, Apricot [106] repairs faulty DNNs by adjusting model weights, and DeepSeer [93] supports interactive RNN debugging through learned state abstraction. None of these techniques observe the runtime behavior of attention components or distinguish among transformer-specific fault categories. As a result, transformer faults that preserve normal loss curves, valid tensor shapes, and finite numerical values fall outside their detection scope, which is the gap DEFault++ targets.

A third category of work uses static analysis to find DL faults before or independent of training. Structural and numerical analyzers (NeuraLint [68], DEBAR [108], NerdBug [35]) check meta-model graphs, tensor abstractions, and API usage. Symbolic and backend-comparison analyzers (DeepCheck [25], CRADLE [72]) test trained networks or compare backends to locate library-level bugs, and BugLab [2] self-supervises bug detection on general source code. FL4Deep [64] occupies a middle position: it combines static and dynamic information through a knowledge graph and ranks candidate root causes over that graph.

These techniques answer a different debugging question and therefore are not direct baselines for DEFault++. Static and library-level analyzers find faults that show up in program text, symbolic constraints, API usage, tensor abstractions, or backend inconsistencies. DEFault++ targets faults that leave the transformer program structurally valid and numerically stable but change how attention components behave during training. A transformer can pass type checks, shape checks, and backend consistency checks while still producing abnormal attention behavior. The graph representations in NeuraLint and FL4Deep also serve a different purpose from the FPG. NeuraLint encodes program structure for pre-training checks, and FL4Deep encodes a static-plus-dynamic knowledge graph for ranking. In contrast, the FPG is a structural prior that marks which other components a perturbation could affect through the transformer's

computation graph. We therefore treat these techniques as complementary and evaluate DEFault++ against dynamic techniques that use the same training-time input.

Transformer-Specific Fault Analysis. The closest existing techniques to DEFault++ analyze transformer-internal behavior, but each targets a narrower failure mode than the taxonomy in Figure 3. ATTNChecker [49] flags NaN or Inf values during attention computation, AtPatch [99] localizes attention-map anomalies at the patch level, FT-Transformer [12] addresses hardware soft-error tolerance for vision transformers, and Bug Attention Probe [86] applies attention probing for code understanding. None of them produces a fault category or a root cause across the transformer block. DEFault++ instead covers the full taxonomy and predicts faults at three levels: detection, category, and root cause. Another direction of transformer research uses input perturbation to study robustness and behavioral invariance [11, 40, 78, 90], including DeepTest [88] for autonomous-driving DNNs. These techniques target behavioral failures exposed by perturbed inputs, whereas DEFault++ targets implementation faults that persist during normal training without input perturbation. Recent LLM-assisted debuggers such as SoapFL [73], LLM4FL [76], ChatDBG [47], and BugReAct [34] reason over source code, execution logs, or bug reports to localize bugs in general software. They are complementary to DEFault++: they read software artifacts after a failure is reported, while DEFault++ reads training-time traces from transformer components and predicts a transformer-specific category and root cause.

Explainability for Fault Diagnosis. Diagnostic output is more useful when it includes the rationale behind each prediction. Existing explanation methods often fall into two families. First family is *post-hoc methods*, which compute explanations after a model has provided a decision (e.g., SHAP [53], DiCE [66], Anchors [77]). They return feature attributions, counterfactual examples, or rule-based explanations from a trained classifier. DEFault [39] leveraged post-hoc explanation in its hierarchical fault diagnosis. The second family consists of inherently interpretable methods that build explanations into the model itself. For example, Prototype networks [85] represent each class as a learned prototype and explain a prediction through similarity to that prototype. Concept-bottleneck models [45] route predictions through human-readable intermediate concepts. In this work, we use an inherently interpretable model whose diagnosis identifies which runtime feature groups distinguish the predicted root cause from the nearest alternative. Thus, the explanation comes directly from the diagnosing model rather than from a separate, post hoc analysis.

11 Conclusion

We presented DEFault++, a transformer-specific hierarchical diagnostic technique for detecting, categorizing, and localizing faults in transformer models. DEFault++ extends the hierarchical diagnosis idea introduced for FFNNs, CNNs, and RNNs by DEFault [39] to transformer architectures, and it reports which feature groups provide the main evidence for each diagnosis. DEFault++ organizes the diagnosis around a Fault Propagation Graph, a structural prior that encodes the dependency paths between transformer components defined by the architecture’s forward and backward equations. To support both training and evaluation, we constructed DEFault-bench, a benchmark of 5,556 labeled instances produced by DEForm, our transformer-specific mutation technique. On DEFault-bench, DEFault++ achieves an F1 of 0.826–0.909 for detection, 0.850–0.868 for categorization, and 0.851–0.867 for hierarchical root-cause diagnosis on both encoder and decoder architectures. In a developer study with 21 practitioners across four debugging scenarios, the repair-action accuracy was 83.3% with DEFault++ assistance and 57.1% without it, and 90% of participants indicated that they would use such a tool in practice.

References

- [1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference. In *Proceedings of the 7th MLSys Conference (MLSys)*.
- [2] Miltiadis Allamanis, Henry Jackson-Flux, and Marc Brockschmidt. 2021. Self-supervised bug detection and repair. In *Advances in Neural Information Processing Systems*, Vol. 34. 27865–27876.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer Normalization. *arXiv preprint arXiv:1607.06450* (2016). doi:10.48550/arXiv.1607.06450
- [4] Houssein Ben Braiek and Foutse Khomh. 2023. Testing feedforward neural networks training programs. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–61. doi:10.1145/3529318
- [5] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonnell, Jason Phang, Samuel Pittman, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2021. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow. *arXiv preprint arXiv:2104.00006* (2021). doi:10.48550/arXiv.2104.00006
- [6] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. 2018. A systematic study of the class imbalance problem in convolutional neural networks. *Neural networks* 106 (2018), 249–259. doi:10.1016/j.neunet.2018.07.011
- [7] Jialun Cao, Meiziniu Li, Xiao Chen, Ming Wen, Yongqiang Tian, Bo Wu, and Shing-Chi Cheung. 2022. Deepfd: Automated fault diagnosis and localization for deep learning programs. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 573–585. doi:10.1145/3510003.3510099
- [8] Rich Caruana. 1997. Multitask Learning. *Machine Learning* 28, 1 (1997), 41–75. doi:10.1023/A:1007379606734
- [9] Gautam Chakrabarti, Vinod Grover, Bastiaan Aarts, Xiangyun Kong, Manjunath Kudlur, Yuan Lin, Jaydeep Marathe, Mike Murphy, and Jian-Zhong Wang. 2012. CUDA: Compiling and optimizing for a GPU platform. *Procedia Computer Science* 9 (2012), 1910–1919. doi:10.1016/j.procs.2012.04.209
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [11] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. 2019. What Does BERT Look at? An Analysis of BERT’s Attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP*. 276–286. doi:10.18653/v1/W19-4828
- [12] Huangliang Dai, Shixun Wu, Jiajun Huang, Zizhe Jian, Yue Zhu, Haiyang Hu, and Zizhong Chen. 2025. FT-Transformer: Resilient and Reliable Transformer with End-to-End Fault Tolerant Attention. *arXiv:2504.02211 [cs.DC]* doi:10.48550/arXiv.2504.02211 arXiv:2504.02211v2.
- [13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, Vol. 35. 16344–16359.
- [14] Alana de Santana Correia and Esther Luna Colombini. 2022. Attention, please! A survey of neural attention models in deep learning. *Artificial Intelligence Review* 55, 8 (2022), 6037–6124. doi:10.1007/s10462-022-10148-x
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies*. 4171–4186. doi:10.18653/v1/N19-1423
- [16] Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas. 2021. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. In *International Conference on Machine Learning*. 2793–2803.
- [17] Hasan Ferit Eniser, Simos Gerasimou, and Alper Sen. 2019. Deepfault: Fault localization for deep neural networks. In *International Conference on Fundamental Approaches to Software Engineering*. Springer International Publishing, Cham, 171–189. doi:10.1007/978-3-030-16722-6_10
- [18] Pál Erdős and Alfréd Rényi. 1959. On Random Graphs I. *Publicationes Mathematicae Debrecen* 6 (1959), 290–297.
- [19] Andre Esteva, Brett Kuprel, Roberto A Novoa, Justin Ko, Susan M Swetter, Helen M Blau, and Sebastian Thrun. 2017. Dermatologist-level classification of skin cancer with deep neural networks. *Nature* 542, 7639 (2017), 115–118. doi:10.1038/nature21056
- [20] Kavin Ethayarajh. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 55–65. doi:10.18653/v1/D19-1006
- [21] Michael G Findley, Kyosuke Kikuta, and Michael Denly. 2021. External validity. *Annual Review of Political Science* 24, 1 (2021), 365–393. doi:10.1146/annurev-polisci-041719-102556
- [22] Gemini Team. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023). doi:10.48550/arXiv.2312.11805
- [23] Aaron Gokaslan and Vanya Cohen. 2019. OpenWebText Corpus. <http://Skylion007.github.io/OpenWebTextCorpus>.
- [24] Phillip Good. 2005. *Permutation, Parametric, and Bootstrap Tests of Hypotheses* (3rd ed.). Springer. doi:10.1007/b138696
- [25] Divya Gopinath, Mengshi Zhang, Kaiyuan Wang, Ismet Burak Kadron, Corina Pasareanu, and Sarfraz Khurshid. 2019. Symbolic execution for importance analysis and adversarial generation in neural networks. In *Proceedings of the IEEE International Symposium on Software Reliability*

- Engineering (ISSRE)*. IEEE, 313–322. doi:10.1109/ISSRE.2019.00039
- [26] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning*. PMLR, 1321–1330.
- [27] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- [28] Qiang Hu, Lei Ma, Xiaofei Xie, Bing Yu, Yang Liu, and Jianjun Zhao. 2019. Deepmutation++: A mutation testing framework for deep learning systems. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1158–1161. doi:10.1109/ASE.2019.00126
- [29] Hugging Face. 2024. Transformers Library. <https://github.com/huggingface/transformers>. Accessed: 2025-04-21.
- [30] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of real faults in deep learning systems. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 1110–1121. doi:10.1145/3377811.3380395
- [31] Nargiz Humbatova, Gunel Jahangirova, and Paolo Tonella. 2021. Deepcrime: mutation testing of deep learning systems based on real faults. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 67–78. doi:10.1145/3460319.3464825
- [32] Ashraf Islam, Benjamin Lundell, Harpreet Sawhney, Sudipta N Sinha, Peter Morales, and Richard J Radke. 2023. Self-supervised learning with local contrastive loss for detection and semantic segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 5624–5633. doi:10.1109/WACV56688.2023.00558
- [33] Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A comprehensive study on deep learning bug characteristics. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 510–520. doi:10.1145/3338906.3338955
- [34] Nifal Islam, Ragib Shahriar Ayon, Deepak George Thomas, Shibbir Ahmed, and Mohammad Wardat. 2026. When Agents Fail: A Comprehensive Study of Bugs in LLM Agents with Automated Labeling. *arXiv preprint arXiv:2601.15232* (2026). doi:10.48550/arXiv.2601.15232
- [35] Foad Jafarnejad, Krishna Narasimhan, and Mira Mezini. 2021. NerdBug: automated bug detection in neural networks. In *Proceedings of the 1st ACM International Workshop on AI and Software Testing/Analysis*. 13–16. doi:10.1145/3464968.3468409
- [36] Sigma Jahan. 2026. DEFault++ Replication Package. <https://github.com/SigmaJahan/DEFaultplusplus-Transformer-Debugging>. Available at: <https://github.com/SigmaJahan/DEFaultplusplus-Transformer-Debugging>.
- [37] Sigma Jahan, Saurabh Rajput, Tushar Sharma, and Mohammad Masudur Rahman. 2026. Why Attention Fails: A Taxonomy of Faults in Attention-Based Neural Networks. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.
- [38] Sigma Jahan, Saurabh Singh Rajput, Tushar Sharma, and Mohammad Masudur Rahman. 2025. Taxonomy of Faults in Attention-Based Neural Networks. *arXiv:2508.04925 [cs.SE]* doi:10.48550/arXiv.2508.04925
- [39] Sigma Jahan, Mehil B Shah, Parvez Mahbub, and Mohammad Masudur Rahman. 2025. Improved Detection and Diagnosis of Faults in Deep Neural Networks Using Hierarchical and Explainable Classification. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 2944–2956. doi:10.1109/ICSE55347.2025.00224
- [40] Erik Jones, Robin Jain, and Percy Liang. 2020. Automatically Finding Bugs in a Classifier: A Systematic Approach. *arXiv preprint arXiv:2003.02907* (2020). *arXiv:2003.02907* doi:10.48550/arXiv.2003.02907
- [41] Hans Kamp and Barbara Partee. 1995. Prototype theory and compositionality. *Cognition* 57, 2 (1995), 129–191. doi:10.1016/0010-0277(94)00659-9
- [42] Prannay Khosla, Yonglong Tian, Huiwen Wang, Ce Liu, Phillip Isola, Dilip Krishnan, Yao Tian, Tatsunori B. Hashimoto, Alan Yuille, Quoc V. Le, Cordelia Schmid, and Trevor Darrell. 2020. Supervised Contrastive Learning. In *Advances in Neural Information Processing Systems*.
- [43] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*.
- [44] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [45] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. 2020. Concept Bottleneck Models. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 5338–5348.
- [46] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. 2019. Similarity of Neural Network Representations Revisited. In *International Conference on Machine Learning*. PMLR, 3519–3529.
- [47] Kyla H. Levin, Nicolas van Kempen, Emery D. Berger, and Stephen N. Freund. 2025. ChatDBG: Augmenting Debugging with Large Language Models. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM. doi:10.1145/3729355
- [48] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3538–3545. doi:10.1609/aaai.v32i1.11604
- [49] Yuhang Liang, Xinyi Li, Jie Ren, Ang Li, Bo Fang, and Jieyang Chen. 2025. ATTNChecker: Highly-Optimized Fault Tolerant Attention for Large Language Model Training. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. doi:10.1145/3710848.3710870
- [50] Shuai Lin, Chen Liu, Pan Zhou, Zi-Yuan Hu, Shuojia Wang, Ruihui Zhao, Yefeng Zheng, Liang Lin, Eric Xing, and Xiaodan Liang. 2022. Prototypical graph contrastive learning. *IEEE transactions on neural networks and learning systems* 35, 2 (2022), 2747–2758. doi:10.1109/TNNLS.2022.3191086
- [51] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Joshi Mandar, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692* (2019). doi:10.48550/arXiv.1907.11692
- [52] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.

- [53] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [54] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. Deepmutation: Mutation testing of deep learning systems. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 100–111. doi:10.1109/ISSRE.2018.00021
- [55] Shiqing Ma, Yingqi Liu, Wen-Chuan Lee, Xiangyu Zhang, and Ananth Grama. 2018. MODE: automated neural network model debugging via state differential analysis and input selection. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 175–186. doi:10.1145/3236024.3236082
- [56] Abdulrahman Mahmoud, Neeraj Aggarwal, Alex Nobbe, Jose Rodrigo Sanchez Vicarte, Sarita V. Adve, Christopher W. Fletcher, Iuri Frosio, and Siva Kumar Sastry Hari. 2020. PyTorchFI: A Runtime Perturbation Tool for DNNs. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops, DSN Workshops 2020, Valencia, Spain, June 29 - July 2, 2020*. IEEE, 25–31. doi:10.1109/DSN-W50199.2020.00014
- [57] Ruchira Manke, Mohammad Wardat, Foutse Khomh, and Hridesh Rajan. 2025. Mock Deep Testing: Toward Separate Development of Data and Models for Deep Learning. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (Ottawa, Ontario, Canada) (ICSE '25)*. IEEE Press, 2970–2982. doi:10.1109/ICSE55347.2025.00220
- [58] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a Large Annotated Corpus of English: The Penn Treebank. *Computational Linguistics* 19, 2 (1993), 313–330.
- [59] Sergei Maslov and Kim Sneppen. 2002. Specificity and Stability in Topology of Protein Networks. *Science* 296, 5569 (2002), 910–913. doi:10.1126/science.1065103
- [60] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. An Empirical Model of Large-Batch Training. *arXiv preprint arXiv:1812.06162* (2018). doi:10.48550/arXiv.1812.06162
- [61] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv preprint arXiv:1609.07843* (2016). doi:10.48550/arXiv.1609.07843 Introduces the WikiText datasets.
- [62] Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one?. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [63] Evan Miller. 2023. Attention Is Off By One. <https://www.evanmiller.org/attention-is-off-by-one.html>. Blog post.
- [64] Mohammad Mehdi Morovati, Amin Nikanjam, and Foutse Khomh. 2024. Fault Localization in Deep Learning-based Software: A System-level Approach. *arXiv preprint arXiv:2411.08172* (2024). arXiv:2411.08172 [cs.SE] doi:10.48550/arXiv.2411.08172
- [65] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. 2021. On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nzpLWnVAYah>
- [66] Ramaravind K. Mothilal, Amit Sharma, and Chenhao Tan. 2020. DiCE: Diverse Counterfactual Explanations for Machine Learning Classifiers. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*. ACM, 607–617. doi:10.1145/3351095.3372850
- [67] Gireen Naidu, Tranos Zuva, and Elias Mmbongeni Sibanda. 2023. A review of evaluation metrics in machine learning algorithms. In *Computer science on-line conference*. Springer, 15–25. doi:10.1007/978-3-031-35314-7_2
- [68] Amin Nikanjam, Housseem Ben Braiek, Mohammad Mehdi Morovati, and Foutse Khomh. 2021. Automatic fault detection for deep learning programs using graph transformations. *ACM Transactions on Software Engineering and Methodology* 31, 1 (2021), 1–27. doi:10.1145/3470006
- [69] NVIDIA. 2021. NVIDIA Deep Learning Profiler (DLProf) User Guide.
- [70] OpenAI. 2023. GPT-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). doi:10.48550/arXiv.2303.08774
- [71] Denis Paperno, German Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Roberto Fernández. 2016. The LAMBADA Dataset: Word Prediction Requiring Broad Discourse Context. In *Proceedings of ACL 2016*. doi:10.18653/v1/P16-1144
- [72] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. 2019. CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1027–1038. doi:10.1109/ICSE.2019.00107
- [73] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1173–1187. doi:10.1109/TSE.2025.3543187
- [74] Ketai Qiu, Niccolò Puccinelli, Matteo Ciniselli, and Luca Di Grazia. 2025. From Today’s Code to Tomorrow’s Symphony: The AI Transformation of Developer’s Routine by 2030. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025). doi:10.1145/3709353
- [75] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. OpenAI blog, 9 pages.
- [76] Md Nakhla Rafi, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. A Multi-Agent Approach to Fault Localization via Graph-Based Retrieval and Reflexion. *arXiv preprint arXiv:2409.13642* (2024). arXiv:2409.13642 [cs.SE] doi:10.48550/arXiv.2409.13642
- [77] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2018. Anchors: High-Precision Model-Agnostic Explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32. doi:10.1609/aaai.v32i1.11491
- [78] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL, 4902–4912. doi:10.18653/v1/2020.acl-main.442
- [79] Lucas Roquet, Fernando Fernandes dos Santos, Paolo Rech, Marcello Traiola, Olivier Sentieys, and Angeliki Kritikakou. 2024. Cross-layer reliability evaluation and efficient hardening of large vision transformers models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

- doi:10.1145/3649329.3655688
- [80] Olivier Roy and Martin Vetterli. 2007. The effective rank: A measure of effective dimensionality. In *2007 15th European signal processing conference*. IEEE, 606–610.
- [81] Sebastian Ruder. 2017. An Overview of Multi-Task Learning in Deep Neural Networks. *arXiv preprint arXiv:1706.05098* (2017). doi:10.48550/arXiv.1706.05098
- [82] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* (2019). doi:10.48550/arXiv.1910.01108
- [83] Eldon Schoop, Forrest Huang, and Björn Hartmann. 2021. UMLAUT: Debugging Deep Learning Programs using Program Structure and Model Behavior. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–16. doi:10.1145/3411764.3445538
- [84] Gregory T Smith. 2005. On construct validity: issues of method and measurement. *Psychological assessment* 17, 4 (2005), 396. doi:10.1037/1040-3590.17.4.396
- [85] Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical Networks for Few-shot Learning. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [86] Adam Stein, Arthur Wayne, Aaditya Naik, Mayur Naik, and Eric Wong. 2025. Where's the Bug? Attention Probing for Scalable Fault Localization. *arXiv preprint arXiv:2502.13966* (2025). doi:10.48550/arXiv.2502.13966
- [87] Florian Tamboon, Foutse Khomh, and Giuliano Antoniol. 2023. A Probabilistic Framework for Mutation Testing in Deep Neural Networks. *Information and Software Technology* 155 (2023), 107129. doi:10.1016/j.infsof.2022.107129 Also available as arXiv:2208.06018 [cs.SE].
- [88] Yuchi Tian, Kexin Pei, Suman Jana, and Baishakhi Ray. 2018. DeepTest: Automated Testing of Deep-Neural-Network-Driven Autonomous Cars. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM, Gothenburg, Sweden, 303–314. doi:10.1145/3180155.3180220
- [89] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. 5998–6008.
- [90] Elena Voita, Jean-Baptiste Talbot, Rico Sennrich Yandex Research Ivan Titov University of Edinburgh Moiseev, and Rico Sennrich. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 5797–5808. doi:10.18653/v1/P19-1580
- [91] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *Proceedings of ICLR 2019*. doi:10.18653/v1/W18-5446
- [92] Feng Wang and Huaping Liu. 2021. Understanding the behaviour of contrastive loss. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2495–2504. doi:10.1109/CVPR46437.2021.00252
- [93] Zhijie Wang, Yuheng Huang, Da Song, Lei Ma, and Tianyi Zhang. 2023. DeepSeer: Interactive RNN Explanation and Debugging via State Abstraction. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. ACM. doi:10.1145/3544548.3580852
- [94] Mohammad Wardat, Breno Dantas Cruz, Wei Le, and Hridayesh Rajan. 2022. Deepdiagnosis: automatically diagnosing faults and recommending actionable fixes in deep learning programs. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 561–572. doi:10.1145/3510003.3510071
- [95] Mohammad Wardat, Breno Dantas Cruz, Wei Le, and Hridayesh Rajan. 2023. An effective data-driven approach for localizing deep learning faults. *arXiv preprint arXiv:2307.08947* (2023). doi:10.48550/arXiv.2307.08947
- [96] Mohammad Wardat, Wei Le, and Hridayesh Rajan. 2021. Deeplocalize: Fault localization for deep neural networks. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 251–262. doi:10.1109/ICSE43902.2021.00034
- [97] Ni Wayan Surya Wardhani, Masithoh Yessi Rochayani, Atiek Iriany, Agus Dwi Sulistyono, and Prayudi Lestantyo. 2019. Cross-validation metrics for evaluating classification performance on imbalanced data. In *Proceedings of the International Conference on Computer, Control, Informatics and Its Applications (IC3INA)*. IEEE, 14–18. doi:10.1109/IC3INA48034.2019.8949568
- [98] Kun Wei, Zhe Xu, and Cheng Deng. 2025. Compress to One Point: Neural Collapse for Pre-Trained Model-Based Class-Incremental Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 21465–21473. doi:10.1609/aaai.v39i20.35448
- [99] Shihao Weng, Yang Feng, Jincheng Li, Yining Yin, Xiaofei Xie, and Jia Liu. 2026. AtPatch: Debugging Transformers via Hot-Fixing Over-Attention. *arXiv preprint arXiv:2601.21695* (2026). doi:10.48550/arXiv.2601.21695
- [100] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. doi:10.1007/978-3-642-29044-2
- [101] Tom Nuno Wolf, Fabian Bongratz, Anne-Marie Rickmann, Sebastian Pölsterl, and Christian Wachinger. 2024. Keep the faith: Faithful explanations in convolutional neural networks for case-based reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 5921–5929. doi:10.1609/aaai.v38i6.28406
- [102] Dangwei Wu, Beijun Shen, Yuting Chen, He Jiang, and Lei Qiao. 2021. Tensfa: detecting and repairing tensor shape faults in deep learning systems. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 11–21. doi:10.1109/ISSRE52982.2021.00014
- [103] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On Layer Normalization in the Transformer Architecture. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR. <http://proceedings.mlr.press/v119/xiong20b.html>
- [104] Ming Yan, Junjie Chen, Xiangyu Zhang, Lin Tan, Gan Wang, and Zan Wang. 2021. Exposing numerical bugs in deep learning via gradient back-propagation. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 100–110. doi:10.1145/3458019.3458030

- 2653 *Engineering (ESEC/FSE)*. 627–638. doi:10.1145/3468264.3468612
- 2654 [105] Shuangfei Zhai, Tatiana Likhomanenko, Etai Littwin, Dan Busbridge, Jason Ramapuram, Yizhe Zhang, Jiatao Gu, and Joshua M. Susskind. 2023.
- 2655 Stabilizing Transformer Training by Preventing Attention Entropy Collapse. In *International Conference on Machine Learning*. 40770–40803.
- 2656 [106] Hao Zhang and W. K. Chan. 2019. Apricot: A Weight-Adaptation Approach to Fixing Deep Learning Models. In *Proceedings of the 34th IEEE/ACM*
- 2657 *International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, CA, USA, 376–387. doi:10.1109/ASE.2019.00043
- 2658 [107] Xiaoyu Zhang, Juan Zhai, Shiqing Ma, and Chao Shen. 2021. Autotrainer: An automatic dnn training problem detection and repair system. In
- 2659 *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 359–371. doi:10.1109/ICSE43902.2021.00043
- 2660 [108] Yuhao Zhang, Luyao Ren, Liqian Chen, Yingfei Xiong, Shing-Chi Cheung, and Tao Xie. 2020. Detecting numerical bugs in neural network
- 2661 architectures. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*
- 2662 *(ESEC/FSE)*. 826–837. doi:10.1145/3368089.3409720

Appendix A DEForm Mutation Operators

DEForm implements 52 mutation operators across the 12 fault categories, listed in full here and separated into attention-specific operators (Table 18) and architecture-level operators (Table 19). 3 introduces the three-letter operator IDs and the Search-Type column.

Table 18. Attention-specific mutation operators used to construct DEForm-bench

Component	Root cause	Operator	ID	Executable Mutation Parameters	ST
Masking	Mask application	Zero attention mask	MZM	Replace valid attention mask with an all-zero mask of the same shape.	B
	Mask application	Invert attention mask	MIM	Invert mask semantics while preserving expected mask shape & dtype.	B
	Mask generation	Reshape mask incorrectly	MRM	<i>axis</i> : batch or head dimension to misalign; retained only if broadcast-compatible.	EL
	Dynamic mask	Causal-mask break (decoder)	MCB	<i>visibility</i> : fraction of future keys made visible within valid mask tensor.	EU
QKV Projection	Parameter initialization	Zero Q projection	QZQ	Zero the existing Q projection weights without changing tensor shape.	B
	Parameter initialization	Zero K projection	QZK	Zero the existing K projection weights without changing tensor shape.	B
	Parameter initialization	Zero V projection	QZV	Zero the existing V projection weights without changing tensor shape.	B
	Head interaction	Swap $Q \leftrightarrow K$	QSW	Swap compatible Q and K projection tensors or outputs.	B
	Head interaction	Tie head weights	QTH	<i>heads</i> : subset of existing heads tied while preserving projection dimensionality.	EL
	Dynamic parameter registration	Freeze QKV gradients	QFG	Disable updates to existing QKV parameters while preserving the forward path.	B
	Dimension	Repartition head dimensions	QHD	<i>split</i> : alternative ($heads, d_{head}$) head factorization with $heads \cdot d_{head} = d_{model}$; retained only when broadcast-compatible.	EL
Score	Normalization	Drop $1/\sqrt{d_k}$ scaling	SDS	Remove score scaling while preserving attention-score shape.	B
	Implementation	Apply dropout pre-softmax	SPD	p : dropout probability applied to score tensors before softmax.	EU
	Precision handling	Unsafe fp16 cast	SUC	Cast score computation to fp16; configurations producing non-finite traces are discarded.	B
Positional	Indexing	Omit positional embeddings	POE	Remove or zero the positional contribution while preserving hidden-state shape.	B
	Relative position	Shift position indices	PSI	Δ : index shift restricted to valid supported positions.	EU
	Interpolation	Truncate positional support	PTL	<i>cutoff</i> : maximum retained position index; out-of-range indices are discarded.	EU
Kernel	Silent fallback	Force slow backend	KSB	Force a valid non-optimized attention backend.	B
	Feature constraints	Mismatched dropout probability	KMD	P_{cfg}, P_{kern} : valid training vs. kernel dropout settings.	EU
	Hardware incompatibility	Trigger valid fallback (dtype/layout)	KFT	<i>dtype</i> or <i>layout</i> mismatch that triggers a supported fallback path.	EL
	Numerical precision	Reduced-precision kernel math	KRP	<i>precision</i> : reduced-precision accumulation mode for the attention kernel. Configurations producing non-finite traces are discarded.	EL
	Kernel memory	Memory-constrained kernel path	KMC	Force the attention kernel onto a reduced-memory chunked execution path. Configurations producing non-finite traces are discarded.	B
Variant	Variant configuration	Single effective head attention	VSH	Route, tie, or mask heads so that only one effective head remains while preserving output shape.	B
	Dynamic dispatch	Causal mask in encoder	VEC	Apply a broadcast-compatible causal mask in an encoder attention layer.	B
KV Cache (decoder)	Cache invalidation	Stale cache	CST	<i>layers</i> : subset of layers using stale but shape-compatible cached K, V states.	EL
	Update synchronization	Desynchronized cache update	CDU	Withhold the current decoding step's freshly computed K, V from the cache so attention and prediction read desynchronized cached states. Cache tensor shape is preserved.	B
	Cache-position	Off-by-one indexing	COB	<i>shift</i> : cache index offset restricted to valid cache positions.	EU
	Memory layout	Truncate cache	CTR	<i>length</i> : maximum retained cache length while preserving the expected cache interface.	EU
	Distributed synchronization	Cross-request leak	CLK	Reuse compatible cached states across requests without changing cache tensor shape.	B

Table 19. Architecture-level mutation operators used to construct DEFault-bench

Component	Root cause	Operator	ID	Executable Mutation Parameters	ST
Embedding	Input initialization	Zero token embedding subset	ETZ	<i>percentage</i> : fraction of vocabulary entries zeroed while preserving embedding shape.	EU
	Input type	Swap embedding pairs	ESW	<i>percentage</i> : fraction of token pairs swapped within existing vocabulary.	EU
	Input type	Scale segment / type embedding	ESS	<i>factor</i> : multiplicative scale applied to existing segment or type embeddings.	EU
	Input dimension	Zero embedding feature dimensions	EZD	<i>fraction</i> : contiguous block of embedding feature dimensions zeroed while embedding tensor shape is preserved.	EU
FFN	Weight scaling	Scale FFN weights	FSW	<i>factor</i> : multiplicative scale on existing W_1, W_2 tensors.	EU
	Neuron dropout	Permanently drop neurons	FDN	<i>percentage</i> : fraction of hidden neurons zeroed while preserving FFN shape.	EU
	Activation function	Change activation function	FCA	<i>activation</i> : executable replacement nonlinearity (e.g., GELU $\rightarrow$ ReLU).	EL
	Regularization	Change weight regularization	FRG	<i>scheme</i> : replacement weight-decay or L_2 coefficient accepted by the optimizer.	EU
	Weight initialization	Change weight initialization	FWI	<i>init</i> : replacement initialization scheme applied to existing W_1, W_2 tensors.	EL
LayerNorm	Scale parameter	Scale γ	NSG	<i>factor</i> : multiplicative scale on existing γ parameters.	EU
	Scale parameter	Zero γ	NZG	Zero existing γ parameters while preserving LayerNorm shape.	B
	Bias parameter	Shift β	NSB	<i>shift</i> : additive offset on existing β parameters.	EU
	Stability parameter	Change ϵ	NCE	<i>value</i> : positive numerical-stability constant accepted by LayerNorm.	EU
	Regularization	Weight-decay LayerNorm parameters	NWD	<i>coefficient</i> : weight-decay coefficient applied to the LayerNorm γ, β parameters, which are normally excluded from decay.	EU
Residual	Skip connection	Remove skip connection	RRS	Zero/bypass residual branch while preserving sublayer output shape.	B
	Residual scaling	Scale residual branch	RSR	<i>factor</i> : multiplicative scale on the residual path.	EU
	Residual path	Inject Gaussian noise	RIN	σ : standard deviation of additive noise with matching tensor shape.	EU
	Gradient clipping	Change gradient clipping	RGC	<i>value</i> : replacement max-norm clip threshold accepted by the training loop.	EU
Output	Dropout rate	Change residual dropout rate	RDR	p : residual-branch dropout probability accepted by the training loop.	EU
	Output scaling	Scale output logits	OSL	<i>factor</i> : multiplicative scale on logits while preserving output dimension.	EU
	Output dimension	Zero rows for selected classes	OZR	<i>classes</i> : subset of existing output rows to zero.	EL
	Output type	Reinitialize output projection	ORI	<i>init</i> : replacement initialization scheme applied to existing output projection.	EL
	Output dimension	Change output interface	OOD	<i>dim/map</i> : root-cause-derived output-dimension or mapping fault; retained only when compatible with the task loss.	EU

Appendix B Fault Injection & Validation

Algorithm 2 Per-configuration fault injection and validation

Require: Clean model θ , configuration C , seeds $\mathcal{S}$

Ensure: Label *isKilled*, instance feature vector $\mathbf{x}$

```

1:  $\theta' \leftarrow \text{Inject}(\theta, C)$  ▷ static or dynamic mutation
2: if VerifyStructural( $\theta, \theta', C$ ) = false then return discard
3: end if
4: for  $s \in \mathcal{S}$  do
5:    $a_s^c \leftarrow \text{TrainAndEvaluate}(\theta, s)$ ;    $a_s^f \leftarrow \text{TrainAndEvaluate}(\theta', s)$ 
6: end for
7:  $p \leftarrow \text{SignFlipPermTest}(\{a_s^c\}, \{a_s^f\})$  ▷ one-sided, paired
8:  $killed \leftarrow [p \leq \alpha]$  ▷ Equation (6)
9:  $\mathbf{x} \leftarrow \text{Aggregate}(\{a_s^c\}, \{a_s^f\})$  ▷ clean-to-faulty deltas
10: return (killed,  $\mathbf{x}$ )

```

Appendix C Per-pair Mutation Scores

The per-pair (i.e., model, task) heatmap (Figure 15) shows the killed mutant rate for all encoder–decoder pairs.

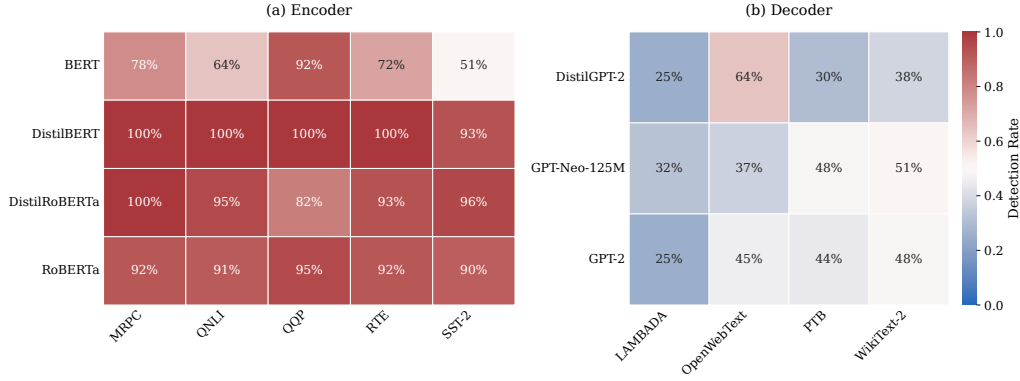


Fig. 15. Mutation scores per (model, task) pair under *isKilled* at $\alpha = 0.05$

Appendix D Fault Propagation Graph Edge Derivation

Each FPG edge is derived from the transformer’s forward and backward equations. The source, target, mechanism class, and governing relation for every edge are listed below (Table 20). 4.1 groups the seven mechanisms into three classes.

Table 20. Edge derivation for the Fault Propagation Graph (FPG)

Source	Target	Mechanism Class	Scope	Details
Embedding	QKV Projection	Forward prop. (M1)	Enc/Dec	$Q, K, V = W_{Q,K,V} E[x]$
Positional	QKV Projection	Forward prop. (M1)	Enc/Dec	Position added to embedding before projection
QKV Proj.	Score Comp.	Simultaneous (M2)	Enc/Dec	$S = QK^T / \sqrt{d_k}$; Q, K from projection
QKV Proj.	Attn. Output	Simultaneous (M2)	Enc/Dec	$\text{Attn} = \text{softmax}(S)$; V from projection
QKV Proj.	KV Cache	Simultaneous (M2)	Dec	K, V stored in cache at each step
Score Comp.	Attn. Mask	Forward prop. (M1)	Enc/Dec	$S' = S + M$
Attn. Mask	Attn. Weights	Forward prop. (M1)	Enc/Dec	$\alpha = \text{softmax}(S')$
Attn. Weights	Attn. Output	Forward prop. (M1)	Enc/Dec	$\text{Attn} = \alpha V$
Attn. Output	Residual (attn)	Forward prop. (M1)	Enc/Dec	$h = x + \text{Attn}(x)$
Residual input	Residual (attn)	Residual bypass (M3)	Enc/Dec	Skip connection: δx passes with unit gain
Residual (attn)	LayerNorm (attn)	Forward prop. (M1)	Enc/Dec	$\hat{x} = \text{LN}(h)$
LayerNorm (attn)	FFN	Forward prop. (M1)	Enc/Dec	FFN input is post-norm representation
FFN	Residual (FFN)	Forward prop. (M1)	Enc/Dec	$h' = h + \text{FFN}(\hat{x})$
Residual input	Residual (FFN)	Residual bypass (M3)	Enc/Dec	Skip connection: δh passes with unit gain
Residual (FFN)	LayerNorm (FFN)	Forward prop. (M1)	Enc/Dec	$\hat{h}' = \text{LN}(h')$
LayerNorm (FFN)	Output Head	Forward prop. (M1)	Enc/Dec	Final representation enters output head
Layer ℓ residual	Layer $\ell+1$	Cross-layer (M4)	Enc/Dec	Repeated M1+M3 across stacked layers
n/a	n/a	Backward prop. (M5)	Enc/Dec	Represented through gradient features
KV Cache	Attn. Weights	Cache-time (M7)	Dec	Cached K, V affect future-step attention
n/a	n/a	Intervention (M6)	Enc/Dec	Joint multi-component effect, represented through fault labels

4

⁴M3 & M4 are forward dependency patterns that expand to multiple directed edges. The diagnostic model represents M5 through gradient features & M6 through fault labels.